Scalable Graph Kernel Query with Error Guarantees

Changan Liu*
College of Computing and Data Science
Nanyang Technological University
Singapore
changan.liu@ntu.edu.sg

Haoyu Liu*
College of Computing and Data Science
Nanyang Technological University
Singapore
haoyu.liu@ntu.edu.sg

Siqiang Luo
College of Computing and Data Science
Nanyang Technological University
Singapore
siqiang.luo@ntu.edu.sg

Laks V.S. Lakshmanan
Department of Computer Science
The University of British Columbia
Canada
laks@cs.ubc.ca

ABSTRACT

We study scalable computing of graph kernel, a fundamental tool for graph similarity with applications in retrieval, provenance, and query processing over graph databases. A prominent instance of this paradigm is the **Random Walk Kernel** (RWK), which is defined upon the Kronecker product of two graphs, capturing fine-grained structural relationships from node pairs. However, existing methods for computing RWK face notable limitations: exact algorithms involve costly matrix operations that scale poorly with graph size, while state-of-the-art (SOTA) approximation methods, while offering linear-time scalability, often provide only heuristic guarantees on accuracy that are not actionable in practice.

We introduce CANWAS, the first algorithm with formal accuracy guarantees, answering an RWK query with target additive error ϵ in $O((\log(1/\epsilon))^2/\epsilon^2)$ time and answering a single-source RWK query in time linear in the input graph size. Building on, yet streamlining, the best prior method, CANWAS truncates the walk series to a logarithmic horizon, sketches hop-wise vectors with fixed-length Rademacher vectors, and reuses two-level transition probability estimates across queries, thereby eliminating redundant random walk sampling. Our new non-trivial theoretical analysis explicitly links walk length, sketch size and failure probability to a tight error bound. Experiments on real-world product graphs with billion-scale nodes show up to a tenfold speed-up over the SOTA, maintaining lighter memory overhead and strong accuracy.

PVLDB Reference Format:
Changan Liu, Haoyu Liu, Siqiang Luo, and Laks V.S. Lakshmanan. Scalable Graph Kernel Query with Error Guarantees. PVLDB, 19(13): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/changanliu/graphKernel.

*Both authors contributed equally to this research.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 13 ISSN 2150-8097.
doi:XX.XX/XXX.XX

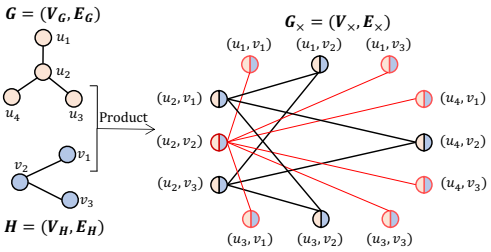


Figure 1: Toy example of a product graph.

1 INTRODUCTION

Graph similarity measurement aims to measure whether two given graphs hold similar patterns [11, 21, 68] and finds various real-world applications including graph machine learning [44, 49, 53], point clouds [4, 41], and chem- and bio-informatics among many others [27, 36, 59, 62]. While traditional measurements such as graph edit distance [58] or subtree patterns [27, 51, 62] offer interpretable criteria, they often struggle to capture global or higher-order topological correspondences [23]. In contrast, *random-walk graph kernels* (RWK) [9, 61, 66, 69] have emerged as an expressive and flexible family of similarity measures based on walk-based alignments.

Given two graphs $G = (V_G, E_G)$, $H = (V_H, E_H)$ with n_G, n_H nodes and m_G, m_H edges respectively, the RWK is defined upon the direct Kronecker product graph [21, 74] $G_\times = (V_\times, E_\times)$, where $V_\times = V_G \otimes V_H$ and two product nodes are adjacent only when their corresponding components are adjacent in both input graphs, as illustrated in Figure 1. Based on the product graph, the typical RWK is calculated compactly as $\kappa(\boldsymbol{v}, \boldsymbol{w}) = \boldsymbol{v}^\top \left[\sum_{\ell=0}^{\infty} \mu_\ell \mathbf{P}_\times^\ell\right] \boldsymbol{w}$, where $\mu_\ell = \alpha(1-\alpha)^\ell$ following classic settings with a fixed decay parameter $\alpha \in (0, 1)$, $\mathbf{P}_\times$ denotes the transition matrix of $G_\times$, and $\mathbf{v}$ (resp. $\mathbf{w}$) describes source node distribution (resp. target weights).

The RWK exhibits strong *modeling power* in graph comparison by counting the number of *common walks* [69]. In graph databases, a query graph G can be compared against each database graph H using RWK scores for retrieval [7, 72, 77, 82**?**], e.g., ranking candidate connectomes in brain graphs [**?**], or candidate shapes in point clouds [4, 41]. This corresponds to *distributional RWK* queries, where $\boldsymbol{v}$ and $\boldsymbol{w}$ summarize sets of candidate node-pair matchings on $G_\times$. Beyond retrieval, RWK has recently been integrated in graph

neural networks [16, 17, 53, 54] where the input graph is compared against a set of learnable "hidden graphs" using RWK, and the resulting kernel similarities are used for downstream prediction. At the same time, RWK can also provide *provenance* for graph search by relating a graph-level match to local node-pair correspondences. A natural approach is to anchor the query at a trusted correspondence $(s,t) \in V_\times$, such as a domain-specified seed match [37] or a matched landmark in two point clouds [60]. The resulting *single-source RWK* assigns each candidate pair a provenance score indicating how strongly it supports the overall RWK similarity.

Expressiveness, however, comes at a computational cost. The textbook approach materializes the product graph and solves the resulting linear system, requiring $O(n_\times^3)$ time and $O(n_\times^2)$ space [21] where $n_\times = n_G n_H$ denotes the number of nodes in $G_\times$. Exploiting the Kronecker structure reduces this cost. For unlabeled graphs, the *Sylvester equation* method requires $O(n^3)$ time and $O(n^2)$ space [6, 68]. Alternatively, iterative methods such as CG and FPI require $O(m^2 i_F)$ time [6]. Here $n = \max\{n_G, n_H\}$, $m = \max\{m_G, m_H\}$, and i_F denotes the number of iterations. However, the computational cost still scales poorly with graph size.

To break the complexity barrier, low-rank and random-feature approximations have been explored. *Ark* [35] uses truncated SVD to construct a low-rank approximation of the transition matrix, with time complexity $O(n^2 + r^6 + mr)$ and memory $O(n^2 r^2 + m)$, but its rank r is chosen heuristically. Recently, GVOYS [15] proposes an unbiased estimator based on lazy random walks and graph-level random features. The method achieves linear time and memory $O(Wn)$ when using a constant number W of walks per node. However, as we discussed in Section 4.1, GVOYS provides no actionable guidance on how large W must be to ensure a given error parameter.

In this paper we present CANWAS (De**C**oupled and Trunc**A**ted Ra**N**dom **WA**lk **S**ampling), the first algorithm that delivers RWK in a resource-constrained setting while providing rigorous guarantees. A RWK query attains an additive-error bound ϵ with high probability in $O((\log(1/\epsilon))^2/\epsilon^2)$ time and $O(1/\epsilon^2)$ space; a single-source RWK query extends gracefully to $O(n(\log(1/\epsilon))^2/\epsilon^2)$ time and $O(n/\epsilon^2)$ space. Our speed and guarantees stem from three ideas. We first truncate the geometric series at $L = O(\log(1/\epsilon))$ hops, proving that the discarded tail contributes at most ϵ to the kernel value. Second, we decouple the product-graph walk into hop-wise transition vectors on G and H, and recouple them with shared Rademacher sketches to obtain an unbiased estimator with explicit variance control. Finally, those vectors are approximated by a two-level sampler that deterministically traverses the very first steps and resorts to lazy random walks only when cheaper. For single-source queries we amortize the computation by re-using lazy random walks across all target pairs, eliminating redundant sampling. The main contributions are summarized as follows.

- **An efficient algorithm with formal error guarantees.** We propose CANWAS that (i) answers a RWK query with target additive error ϵ in $O((\log(1/\epsilon))^2/\epsilon^2)$ time and (ii) evaluates all targets from a fixed source in time linear in the input graph size, while providing rigorous error bounds. This breakthrough significantly advances scalability without compromising accuracy.
- **Comprehensive theoretical analysis.** We thoroughly analyze both the approximation error and computational efficiency of CANWAS. Our theoretical results provide explicit bounds on the trade-off between runtime and accuracy, offering clear guidance for practical parameter tuning.
- **Extensive empirical validation.** We conduct comprehensive experiments on diverse real-world graphs, demonstrating that CANWAS achieves remarkable speed-ups over baselines—while consistently maintaining exceptional accuracy and light memory overhead, validating the practical impact of our approach.

2 PRELIMINARIES

2.1 Notations

Let $G = (V, E)$ be an undirected graph with node set V and edge set $E \subseteq V \times V$. Let $\mathcal{N}(u)$ denote the neighbor set of node u, and $\boldsymbol{d}_u = |\mathcal{N}(u)|$ its degree. We denote by $\mathbf{A} \in \{0,1\}^{n\times n}$ the adjacency matrix whose entry $\mathbf{A}_{ij} = 1$ if $(i,j) \in E$ and $\mathbf{A}_{ij} = 0$ otherwise, and $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ the transition matrix, where $\mathbf{D} = \mathrm{diag}(\boldsymbol{d}_1, \cdots, \boldsymbol{d}_n)$. The ℓ-th power of the transition matrix $\mathbf{P}$ describes the probability of length ℓ walks. We denote by $p^\ell(s,t)$ the probability of a transition from vertex s to vertex t via a walk of length ℓ. We use $\boldsymbol{e}_i$ to denote the i-th standard basis (i.e., a vector of all zeros with the i-th entry set to one) and $\mathbf{I}$ to denote the identity matrix. When it is clear from context, we leave the dimensions of vectors and matrices implicit. We denote by $\otimes$ the Kronecker product: for matrices $\mathbf{M} \in \mathbb{R}^{n\times m}$ and $\mathbf{N} \in \mathbb{R}^{p\times q}$, the Kronecker product $\mathbf{M} \otimes \mathbf{N} \in \mathbb{R}^{(np)\times(mq)}$ is the block matrix whose (i,j)-th block is $\mathbf{M}_{ij}\mathbf{N}$.

2.2 Product Graph and Random Walk Kernel

Product Graph [21, 74]. Given graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$, their *product* is the graph $G_\times = (V_\times, E_\times)$ where $V_\times = \{(u,v) : u \in V_G \wedge v \in V_H\}$, $E_\times = \{((u_1,v_1),(u_2,v_2)) \mid (u_1,u_2) \in E_G \wedge (v_1,v_2) \in E_H\}$. Its adjacency and transition matrices satisfy $\mathbf{A}_\times = \mathbf{A}_G \otimes \mathbf{A}_H$, and $\mathbf{P}_\times = \mathbf{P}_G \otimes \mathbf{P}_H$, such that $\mathbf{P}_\times((s,t),(u,v)) = \mathbf{P}_G(s,u)\,\mathbf{P}_H(t,v)$. Therefore, a standard random walk on the product graph precisely simulates taking one step simultaneously in G and in H [24].

Random Walk Kernel (RWK) [21, 69] . For two graphs G, H, choose vectors $\boldsymbol{v}, \boldsymbol{w} \in [0,1]^{|V_G|\cdot|V_H|}$ for encoding starting probability distributions and target weights, respectively, and let $\{\mu_\ell\}_{\ell\geq 0}$ be nonnegative weights. RWK is defined as $\kappa(\boldsymbol{v},\boldsymbol{w}) = \boldsymbol{v}^\top\left[\sum_{\ell=0}^\infty \mu_\ell \mathbf{P}_\times^\ell\right]\boldsymbol{w}$. For a fixed decay parameter $\alpha \in (0,1)$, we adopt the standard geometric weights $\mu_\ell = \alpha(1-\alpha)^\ell$. The resulting series converges to

$$\kappa(\boldsymbol{v},\boldsymbol{w}) = \boldsymbol{v}^\top\left[\alpha\big(\mathbf{I} - (1-\alpha)\mathbf{P}_\times\big)^{-1}\right]\boldsymbol{w}. \tag{1}$$

When $\boldsymbol{v}$ and $\boldsymbol{w}$ are indicator vectors, i.e., $\boldsymbol{v} = \boldsymbol{e}_{s,t}$ and $\boldsymbol{w} = \boldsymbol{e}_{u,v}$ satisfying $s,u \in V_G$ and $t,v \in V_H$, we obtain a single-pair RWK

$$\begin{aligned}\kappa(\boldsymbol{e}_{s,t},\boldsymbol{e}_{u,v}) &= \boldsymbol{e}_{s,t}^\top\left[\alpha\big(\mathbf{I} - (1-\alpha)\mathbf{P}_\times\big)^{-1}\right]\boldsymbol{e}_{u,v}\\ &= \sum\nolimits_{\ell=0}^\infty \alpha(1-\alpha)^\ell \mathbf{P}_\times^\ell((s,t),(u,v)).\end{aligned} \tag{2}$$

2.3 Problem Definition

Below, we give the formal definitions for the studied problems.

(ϵ, p_f)-approximation of RWK query. Given an error parameter $\epsilon \in (0,1)$, a source distribution $\boldsymbol{v}$ over $V_\times$, and a target weight vector $\boldsymbol{w} \in [0,1]^{|V_\times|}$, a (ϵ, p_f)-approximation of the RWK query returns an estimate $\widetilde{\kappa}(\boldsymbol{v},\boldsymbol{w})$ such that $|\kappa(\boldsymbol{v},\boldsymbol{w}) - \widetilde{\kappa}(\boldsymbol{v},\boldsymbol{w})| \leq \epsilon$ holds with probability at least $1 - p_f$.

Table 1: Asymptotic complexity of distributional and single-source graph-kernel queries. Abbreviations: SY, CG, FPI, Ark, and GVOYS are defined in the text. i_F is the iteration count in CG/FPI, r the low rank in Ark ($r \ll n_G$), W, a and d are the number of walks, the number of anchor nodes and the number of feature blocks in GVOYS. In this table, we assume $n_G = n_H = n$ and $m_G = m_H = m$ without loss of generality.

Method	Preprocess Time	Query Time		Space Overhead		Guarantee
		distribution	single-source	distribution	single-source	
Direct [21]	$O(n^6)$	$O(1)$	$O(n^2)$	$O(n^4)$	$O(n^4)$	exact
SY [6, 68]	–	$O(n^3)$	$O(n^5)$	$O(n^2)$	$O(n^2)$	exact
CG [6]	–	$O(m^2 i_F)$	$O(n^2 m^2 i_F)$	$O(m^2)$	$O(m^2)$	exact
FPI [6]	–	$O(m^2 i_F)$	$O(n^2 m^2 i_F)$	$O(m^2)$	$O(m^2)$	exact
Ark [35]	$O(n^2 r^4 + r^6 + mr)$	$O(nr + r^4)$	$O(n^3 r + n^2 r^4)$	$O(m + n^2 r^2)$	$O(m + n^2 r^2)$	–
GVOYS [15]	$O(\frac{1}{\alpha} W d n \log a + W d n a)$	$O(ad)$	$O(nad)$	$O(Wnad)$	$O(Wnad)$	–
CANWAS (ours)	–	$O((\log(1/\epsilon))^2/\epsilon^2)$	$O(n(\log(1/\epsilon))^2/\epsilon^2)$	$O(1/\epsilon^2)$	$O(n/\epsilon^2)$	additive ϵ

(ϵ, p_f)-approximation of single-source RWK query. Given two graphs G and H, an error parameter $\epsilon \in (0, 1)$, and a source node $(s, t) \in V_\times$, a (ϵ, p_f)-approximation of single-source RWK estimates $\widetilde{\kappa}(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})$ for every target node $(u, v) \in V_\times$ such that $\Pr[\max_{(u,v)\in V_\times} |\widetilde{\kappa}(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v}) - \kappa(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})| \le \epsilon] \ge 1 - p_f$.

3 EXISTING METHODS

Existing work on RWK can be split into two categories. Table 1 shows the complexity of different approaches. A discussion of broader related literature is deferred to Section 7.

3.1 Exact Methods

The simplest exact recipe materializes the direct-product graph $G_\times$ [21]; just inverting its adjacency matrix will overwhelm the memory (needs $O(n_\times^2)$) once the product graph contains at least 10000 nodes on standard machines. More refined exact solvers avoid building $G_\times$ but remain expensive. All exact accelerations hinge on the following Kronecker–vector identity.

Lemma 3.1 ([6]). *For any* $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{B} \in \mathbb{R}^{n\times p}, \mathbf{C} \in \mathbb{R}^{p\times q}$, $\mathrm{vec}(\mathbf{ABC}) = (\mathbf{C}^\top \otimes \mathbf{A})\ \mathrm{vec}(\mathbf{B})$, *where* $\mathrm{vec}(\mathbf{A})$ *is the vector obtained by stacking the columns of* $\mathbf{A}$.

Below we show how Lemma 3.1 turns the $n_\times \times n_\times$ matrix into two ordinary sparse multiplies, yielding three classical solvers.

Sylvester–equation method (SY) [6, 68]. The RWK is obtained from $\kappa(\boldsymbol{v}, \boldsymbol{w}) = \alpha \boldsymbol{v}^\top \boldsymbol{x}$ where

$$(\mathbf{I} - (1-\alpha)\mathbf{P}_\times)\boldsymbol{x} = \boldsymbol{w}. \tag{3}$$

Rewriting Eq. (3) yields that $\boldsymbol{x} = \boldsymbol{w} + (1-\alpha)\mathbf{P}_\times \boldsymbol{x}$. We write $\boldsymbol{x} = \mathrm{vec}(\mathbf{X})$ and $\boldsymbol{w} = \mathrm{vec}(\mathbf{W})$. Then, $\mathrm{vec}(\mathbf{X}) = (1-\alpha)\mathbf{P}_\times \mathrm{vec}(\mathbf{X}) + \mathrm{vec}(\mathbf{W})$. Based on Lemma 3.1, we have $\mathrm{vec}(\mathbf{X}) = (1-\alpha)\ \mathrm{vec}(\mathbf{P}_G \mathbf{X} \mathbf{P}_H) + \mathrm{vec}(\mathbf{W})$. Removing the $\mathrm{vec}(\cdot)$ operation, then

$$\mathbf{X} = (1-\alpha)\mathbf{P}_G \mathbf{X} \mathbf{P}_H + \mathbf{W},$$

which is a Sylvester equation that costs $O(n^3)$ time [20]. Then, the kernel can be obtained as $\boldsymbol{v}^\top \mathrm{vec}(\mathbf{X})$ in prohibitive $O(n^2)$ time.

Conjugate-gradient method (CG) [6]. Given a matrix $\mathbf{M}$ and a vector $\boldsymbol{b}$, CG solves the system of $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$ efficiently [55]. The CG for graph kernel also focuses on efficient ways to solve equation (3). Again based on Lemma 3.1, let $\boldsymbol{r} = \mathrm{vec}(\mathbf{R})$, one can write

$$\mathbf{P}_\times \boldsymbol{r} = (\mathbf{P}_G \otimes \mathbf{P}_H)\,\mathbf{R} = \mathrm{vec}(\mathbf{P}_G \mathbf{R} \mathbf{P}_H).$$

Each CG iteration takes $O(m^2)$ time for the matrix-vector multiplication. Thus CG takes $O(m^2 i_F)$ time where i_F denotes the number of iterations. A bottleneck of CG is its $O(m^2)$ memory requirement.

Fixed-point iterations (FPI) [6]. FPI begins by rewriting equation (3) as $\boldsymbol{x} = \boldsymbol{w} + (1-\alpha)\mathbf{P}_\times \boldsymbol{x}$ and solve for $\boldsymbol{x}$ by iteratively finding a fixed point of the above equation [55]. Denote $\boldsymbol{x}_t$ the value of $\boldsymbol{x}$ at iteration t, set $\boldsymbol{x}_0 := \boldsymbol{w}$ and compute $\boldsymbol{x}_{t+1} = \boldsymbol{w} + (1-\alpha)\mathbf{P}_\times \boldsymbol{x}_t$ repeatedly until $\|\boldsymbol{x}_{t+1} - \boldsymbol{x}_t\| < \epsilon$. Since each iteration involves computation of the matrix-vector product $\mathbf{P}_\times \boldsymbol{x}_t$, all speed-ups for computing the matrix-vector product discussed above are also applicable here. However, FPI still takes $O(m^2 i_F)$ time for i_F iterations, along with a prohibitive with $O(m^2)$ memory overhead.

Remark. While the exact methods provide the strongest error guarantees, their prohibitive computing and heavy memory overhead greatly limit their scalability and applicability.

3.2 Approximate Methods

Approximation methods improve efficiency by relaxing exactness, but error guarantees of current solutions remain unclear.

Low-rank matrix approximation method. Ark [35] uses a low-rank approximation for the RWK. Their core idea is to use the top-r SVD of $\mathbf{P}_G$ and $\mathbf{P}_H$ to approximate $\mathbf{P}_\times$ as $\hat{\mathbf{P}}_\times = \mathbf{U}_G \Lambda_G \mathbf{V}_G^\top \otimes \mathbf{U}_H \Lambda_H \mathbf{V}_H^\top$, where $\mathbf{U}_G, \mathbf{V}_G \in \mathbb{R}^{n\times r}$ contain the top-r left/right singular vectors of $\mathbf{P}_G$, and the diagonal matrix $\Lambda_G \in \mathbb{R}^{r\times r}$ contains the corresponding singular values (and analogously for H). However, Ark still requires $O(n^2 r^4 + mr + r^6)$ computing time complexity and $O(m + n^2 r^2)$ memory overhead. Meanwhile, r needs to be manually tuned, and [35] offers no theoretical guidance on its choice.

Graph random feature based method. GVOYS [15] constructs graph-level random features via random walk encodings. For each graph G_i with n_i nodes, by choosing a_i anchor nodes it constructs a feature matrix $\mathbf{C}_i \in \mathbb{R}^{n_i \times a_i}$, where each row corresponds to a node feature $\widehat{\phi}_{G_i}(v_j)$ generated by performing W random walks. The walks are then aligned by Rademacher signs g.

To de-correlate estimation, an independent copy $\mathbf{D}_i \in \mathbb{R}^{n_i \times a_i}$ is generated using a separate batch of walks and independent g samples. Then by basic matrix algebra,

$$\kappa(\boldsymbol{v}, \boldsymbol{w}) = \mathbb{E}[(\boldsymbol{v}_G^\top \mathbf{C}_G)(\mathbf{D}_G^\top \boldsymbol{w}_G) \cdot (\boldsymbol{v}_H^\top \mathbf{C}_H)(\mathbf{D}_H^\top \boldsymbol{w}_H)].$$

Each term in parentheses is a scalar computable in $O(n_i r_i)$ time. To obtain graph-level features, D independent blocks $\{\mathbf{C}_i^{(k)}, \mathbf{D}_i^{(k)}\}_{k=1}^{D}$ are sampled, where the parameter D is empirically set. The explicit feature vector $\phi(G) \in \mathbb{R}^D$ is then constructed such that its k-th coordinate is defined as

$$[\phi(G)]_k := \tfrac{1}{\sqrt{D}}\big(\boldsymbol{v}_G^\top \mathbf{C}_G^{(k)}\big)\big((\mathbf{D}_G^{(k)})^\top \boldsymbol{w}_G\big), \quad \forall k \in \{1, 2, \ldots, D\}.$$

These graph fingerprints satisfy $\mathbb{E}[\phi(G)^\top \phi(H)] = \kappa(\boldsymbol{v}, \boldsymbol{w})$, enabling kernel estimation with linear memory and time cost.

However, while GVOYS is unbiased in expectation, it suffers from weak theoretical control over variance. The authors do not provide bounds on the number of walks W, anchor nodes a, or repetitions d_G required to achieve a target accuracy ϵ, leaving practitioners without concrete guidance for parameter selection. As a result, the method often necessitates thousands of walks to suppress variance on large graphs, significantly hindering its computational efficiency.

Remark. Current approximation methods for graph kernels remain in an early stage, lacking either practically useful theoretical guarantees or efficient time and space complexity.

4 THE CANWAS ALGORITHM

4.1 High-level ideas

As discussed in Section 3.2, although GVOYS achieves linear-time scalability, its theoretical guarantees are difficult to translate into practical parameter choices. First, GVOYS samples randomly terminated walks with unbounded lengths, without exploiting the fact that sufficiently long walks have negligible contributions under geometric decay. Second, its concentration bound is conditioned on the shared random variables and does not separately control all sources of approximation error, making it difficult to determine the required parameters. Third, GVOYS constructs random features and compresses them through anchor sampling, introducing intermediate representations and additional tunable parameters. In contrast, our method explicitly controls the different error sources and directly constructs sketches with an end-to-end error guarantee.

First, the geometric decay $\mu_\ell = \alpha(1-\alpha)^\ell$ allows us to truncate the infinite walk series after $L = O(\log(1/\epsilon_t))$ steps for fixed α. We show that the discarded tail contributes at most ϵ_t to the kernel value. This converts the infinite-horizon RWK into a finite sequence of hop-wise transition contributions while introducing a deterministic, explicitly controlled error.

Next, to avoid materializing the dense product graph $G_\times$, we *decouple* the joint dynamics into two independent random processes on the original graphs G and H, inspired by the technique in [15]. Specifically, we incorporate *fixed-length* Rademacher variables [26], which *align* the random walk steps on G and H. By sampling T shared Rademacher vectors $\boldsymbol{g}$ independently, we derive a new unbiased estimator for $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$: $\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) = \frac{1}{T}\sum_{\tau=1}^{T} f_\tau(\cdot)$, where $f_\tau(\cdot)$ is a function involving transition probabilities and the sampled Rademacher vectors, formally defined in Eq. (7). In Lemma 4.4, we prove that this estimator approximates $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$ within an ϵ_r additive error with high probability, provided T is chosen appropriately.

Note that computing $f_\tau(\cdot)$ requires explicitly evaluating multi-hop transition probabilities, which can be computationally expensive. To address this, we propose a further randomized estimator by simulating W_G and W_H lazy random walks. For each walk, we record the stopping node and apply a length-dependent Rademacher weight to combine with the walk outcome. By setting W_G and W_H appropriately, our final estimator, denoted $\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})$, aggregates these weighted walk statistics and still guarantees an ϵ_w additive error, as established in Lemma 4.6. To further accelerate the lazy walk sampling process, we adopt a greedy strategy that performs deterministic graph traversal for the first few steps and switches to lazy walk sampling once the estimated cost exceeds the sampling overhead. This refinement significantly reduces the number of required random walks and improves computational efficiency.

Putting these components together, we obtain a final estimator that, with probability at least $1-p_f$, provides an additive ϵ-accurate estimate of the true kernel value $\kappa(\boldsymbol{v}, \boldsymbol{w})$, where $\epsilon = \epsilon_t + \epsilon_r + \epsilon_w$. Importantly, the estimator achieves a runtime of $O((\log(1/\epsilon))^2/\epsilon^2)$ and a memory usage of $O(\frac{1}{\epsilon^2})$, both independent of the size of the product graph. In the remainder of this section, we detail each step of the algorithm and provide the corresponding theoretical analysis.

4.2 Truncated RWK

In this section, we derive the concept of a *truncated* RWK, which plays an essential role in designing our algorithms.

Recall that RWK is defined as $\kappa(\boldsymbol{v}, \boldsymbol{w}) = \boldsymbol{v}^\top\left[\alpha\big(\mathbf{I}-(1-\alpha)\mathbf{P}_\times\big)^{-1}\right]\boldsymbol{w}$, where $\mathbf{P}_\times = \mathbf{P}_G \otimes \mathbf{P}_H$ is the transition matrix of the product graph. Expanding the resolvent as a Neumann series yields

$$\kappa(\boldsymbol{v}, \boldsymbol{w}) = \sum\nolimits_{\ell=0}^{\infty} \alpha(1-\alpha)^\ell\, \boldsymbol{v}^\top \mathbf{P}_\times^\ell \boldsymbol{w}.$$

The decay factor $\alpha(1-\alpha)^\ell$ guarantees that the influence of terms diminishes geometrically as ℓ increases, thereby ensuring convergence of the sum and emphasizing the dominance of lower-order terms in determining the RWK. This structure naturally motivates the definition of a truncated approximation $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$,

$$\kappa_L(\boldsymbol{v}, \boldsymbol{w}) = \sum\nolimits_{\ell=0}^{L} \alpha(1-\alpha)^\ell\, \boldsymbol{v}^\top \mathbf{P}_\times^\ell \boldsymbol{w}. \tag{4}$$

By retaining $L+1$ terms, $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$ approximates $\kappa(\boldsymbol{v}, \boldsymbol{w})$ by preserving its essential structure. The following lemma ensures that, if L is set appropriately, we obtain an $(\epsilon_t, 0)$-approximation of the RWK.

Lemma 4.1. *Given a source distribution $\boldsymbol{v}$ over $V_\times$, a target weight vector $\boldsymbol{w} \in [0,1]^{|V_\times|}$, and an error $\epsilon_t \in (0,1)$, we have $|\kappa(\boldsymbol{v}, \boldsymbol{w}) - \kappa_L(\boldsymbol{v}, \boldsymbol{w})| \le \epsilon_t$ whenever $L \ge \lceil \log_{1-\alpha} \epsilon_t - 1 \rceil$. For fixed $\alpha \in (0,1)$, this gives $L = O(\log(1/\epsilon_t))$.*

Proof. Since $\mathbf{P}_\times^\ell$ is a row-stochastic transition matrix, $\boldsymbol{v}$ is a probability vector, and $\boldsymbol{w} \in [0,1]^{|V_\times|}$, each inner product satisfies $0 \le \boldsymbol{v}^\top \mathbf{P}_\times^\ell \boldsymbol{w} \le 1$. Hence the truncation error is bounded by

$$|\kappa(\boldsymbol{v}, \boldsymbol{w}) - \kappa_L(\boldsymbol{v}, \boldsymbol{w})| \le \sum\nolimits_{\ell=L+1}^{\infty} \alpha(1-\alpha)^\ell = \alpha\tfrac{(1-\alpha)^{L+1}}{1-(1-\alpha)} = (1-\alpha)^{L+1}.$$

To ensure the error is at most ϵ_t, it suffices to let $L \ge \log_{1-\alpha} \epsilon_t - 1 = O(\log(1/\epsilon_t))$, which yields the stated condition. □

While truncating the infinite sum provides an effective approximation of $\kappa(\boldsymbol{v}, \boldsymbol{w})$, a significant challenge remains: explicitly forming $\mathbf{P}_\times$ incurs quadratic time and space complexity, even for sparse inputs, which severely limits scalability. To address this, we next introduce a decoupling strategy that allows us to approximate the contributions on G and H independently and then recouple them, thereby avoiding materializing the product graph explicitly.

4.3 Decoupling with Rademacher sketches

The method of [15] first *decouples* the computation by simulating random walks on G and H independently, and then *recouples* the results with a set of *variable-length* sketches. However, the sketches have random lengths, making it difficult to apply typical concentration bounds to derive explicit worst-case error guarantees. We keep the similar decoupling idea, but replace the variable-length sketches with a *fixed-length* Rademacher sketch. This allows us to invoke standard Hoeffding-type concentration and, for the first time, obtain explicit high-probability error bounds. The following lemma initiates our decoupling analysis.

Lemma 4.2. *Let $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$ be the length–L truncated RWK defined in Eq. (4). Set the weights $w_\ell = \sqrt{\alpha}\,(1-\alpha)^{\ell/2}$, $\ell = 0, \dots, L$. Then*

$$\kappa_L(\boldsymbol{v}, \boldsymbol{w}) = \sum_{\ell,\ell'=0}^{L} w_\ell \boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G \otimes w_{\ell'} \boldsymbol{v}_H^\top \mathbf{P}_H^{\ell'} \boldsymbol{w}_H \mathbb{I}(\ell, \ell'), \quad (5)$$

where $\mathbb{I}(\ell, \ell')$ equals 1 when $\ell = \ell'$ and equals 0 otherwise.

Proof. Using the Kronecker structure $\boldsymbol{v} = \boldsymbol{v}_G \otimes \boldsymbol{v}_H$, $\boldsymbol{w} = \boldsymbol{w}_G \otimes \boldsymbol{w}_H$ and the property of the Kronecker product [57], we obtain

$$\begin{aligned}\boldsymbol{v}^\top \mathbf{P}_\times^\ell \boldsymbol{w} &= (\boldsymbol{v}_G^\top \otimes \boldsymbol{v}_H^\top)(\mathbf{P}_G \otimes \mathbf{P}_H)^\ell (\boldsymbol{w}_G \otimes \boldsymbol{w}_H) \\ &= (\boldsymbol{v}_G^\top \otimes \boldsymbol{v}_H^\top)(\mathbf{P}_G^\ell \otimes \mathbf{P}_H^\ell)(\boldsymbol{w}_G \otimes \boldsymbol{w}_H) = (\boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G)\,(\boldsymbol{v}_H^\top \mathbf{P}_H^\ell \boldsymbol{w}_H).\end{aligned}$$

Substituting this back into the definition of $\kappa_L(\boldsymbol{v}, \boldsymbol{w})$ yields

$$\kappa_L(\boldsymbol{v}, \boldsymbol{w}) = \sum_{\ell=0}^{L} \alpha(1-\alpha)^\ell\,(\boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G)\,(\boldsymbol{v}_H^\top \mathbf{P}_H^\ell \boldsymbol{w}_H).$$

Finally, we use $w_\ell^2 = \alpha(1-\alpha)^\ell = w_\ell \sum_{\ell'=0}^{L} w_{\ell'}\,\mathbb{I}(\ell, \ell')$ to re-index the sum and naturally contain Eq. (5), completing the proof. □

Decoupled estimator. Lemma 4.2 implies the power series on G and H nearly factorize but require strict order matching. We relax this constraint using a *Rademacher vector* $\boldsymbol{g} \in \{\pm 1\}^{L+1}$, where the correlation property $\mathbb{E}[\boldsymbol{g}(\ell)\boldsymbol{g}(\ell')] = \mathbb{I}(\ell = \ell')$ elegantly softly aligns the terms. Averaging over T independent vectors produces an unbiased estimator with variance decreasing proportionately to $1/T$. We next define this estimator.

Fix a truncation depth L, define for each $\ell = 0, \dots, L$

$$a_\ell := \boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G, \quad b_\ell := \boldsymbol{v}_H^\top \mathbf{P}_H^\ell \boldsymbol{w}_H.$$

We form the vectors

$$\boldsymbol{p}_G := (w_0 a_0, \dots, w_L a_L)^\top, \quad \boldsymbol{p}_H := (w_0 b_0, \dots, w_L b_L)^\top.$$

Draw T i.i.d. Rademacher vectors $\boldsymbol{g}_\tau \in \{\pm 1\}^{L+1}$, $\tau = 1, \dots, T$ and project the fingerprints down to one dimension by inner product:

$$z_{G,\tau} := \langle \boldsymbol{p}_G, \boldsymbol{g}_\tau \rangle, \quad z_{H,\tau} := \langle \boldsymbol{p}_H, \boldsymbol{g}_\tau \rangle. \quad (6)$$

We estimate the truncated RWK by

$$\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) := \frac{1}{T} \sum_{\tau=1}^{T} z_{G,\tau} z_{H,\tau}. \quad (7)$$

Lemma 4.3 (Unbiasedness). *For any product-form query vectors $\boldsymbol{v}$ and $\boldsymbol{w}$, the estimator (7) is unbiased for the truncated RWK kernel defined in Lemma 4.2, that is $\mathbb{E}[\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})] = \kappa_L(\boldsymbol{v}, \boldsymbol{w})$.*

Proof. Because the T trials are i.i.d., it is enough to consider a single Rademacher vector, say $\boldsymbol{g}_1$. Expanding the product gives

$$z_{G,1} z_{H,1} = \langle \boldsymbol{p}_G, \boldsymbol{g}_1 \rangle \langle \boldsymbol{p}_H, \boldsymbol{g}_1 \rangle = \sum_{\ell=0}^{L} \sum_{\ell'=0}^{L} w_\ell w_{\ell'}\, a_\ell b_{\ell'}\, \boldsymbol{g}_1(\ell) \boldsymbol{g}_1(\ell').$$

The coordinates of a Rademacher vector are independent and satisfy $\mathbb{E}[\boldsymbol{g}_1(\ell)\,\boldsymbol{g}_1(\ell')] = \mathbb{I}(\ell, \ell')$. Therefore all cross terms with $\ell \neq \ell'$ vanish in expectation, yielding $\mathbb{E}[z_{G,1} z_{H,1}] = \sum_{\ell=0}^{L} w_\ell^2\, a_\ell b_\ell$. By Lemma 4.2 we have $\kappa_L(\boldsymbol{v}, \boldsymbol{w}) = \sum_{\ell=0}^{L} w_\ell^2\, a_\ell b_\ell$, so that $\mathbb{E}[z_{G,1} z_{H,1}] = \kappa_L(\boldsymbol{v}, \boldsymbol{w})$. Averaging over the T i.i.d. trials preserves the expectation, hence $\mathbb{E}[\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})] = \kappa_L(\boldsymbol{v}, \boldsymbol{w})$, which completes the proof. □

Algorithm 1: Lazy-Walk($G, \boldsymbol{v}_G, p, R$)

Input: Graph G; source distribution $\boldsymbol{v}_G$; stopping probability p; maximal retained length R
Output: Endpoint x; walk length k; validity indicator valid

```
Sample x ~ v_G
k ← 0; valid ← 0
while true do
    if rand(0, 1) ≤ p then valid ← 1; break
    if k = R then break
    x ← randomly sample a neighbor in N(x)
    k ← k + 1
return (x, k, valid)
```

Beyond expectation. Unbiasedness alone does not tell us how many sketches T we need for the estimator to be accurate. A key advantage of the shared Rademacher construction is that the weighted hop profiles have bounded ℓ_2 norm, while the random projection preserves their inner product with high probability.

Lemma 4.4. *For $\epsilon_r \in (0, 1)$ and $p_f \in (0, 1)$, if the sketch number satisfies $T \geq 12\,\epsilon_r^{-2} \log(\frac{4}{p_f})$, then $\Pr[|\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) - \kappa_L(\boldsymbol{v}, \boldsymbol{w})| > \epsilon_r] \leq p_f$.*

Proof. Since $\boldsymbol{v}_G$ and $\boldsymbol{v}_H$ are probability distributions, $\boldsymbol{w}_G$ and $\boldsymbol{w}_H$ have entries in $[0, 1]$, and $\mathbf{P}_G^\ell$ and $\mathbf{P}_H^\ell$ are row-stochastic, we have $0 \leq a_\ell, b_\ell \leq 1$. Hence, $\|\mathbf{p}_G\|_2^2 = \sum_{\ell=0}^{L} \alpha(1-\alpha)^\ell a_\ell^2 \leq \sum_{\ell=0}^{L} \alpha(1-\alpha)^\ell \leq 1$, and symmetrically $\|\mathbf{p}_H\|_2 \leq 1$. Let $\mathbf{R} \in \mathbb{R}^{T \times (L+1)}$ be the normalized Rademacher matrix whose entries are $R_{\tau,\ell} = \frac{\boldsymbol{g}_\tau(\ell)}{\sqrt{T}}$. By the definitions of $z_{G,\tau}$ and $z_{H,\tau}$, $\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) = \langle \mathbf{R}\mathbf{p}_G, \mathbf{R}\mathbf{p}_H \rangle$, whereas Lemma 4.2 gives $\kappa_L(\boldsymbol{v}, \boldsymbol{w}) = \langle \mathbf{p}_G, \mathbf{p}_H \rangle$.

For any fixed vector $\boldsymbol{x}$ and any $\delta \in (0, 1)$, the standard concentration bound for a normalized Rademacher projection [1] gives

$$\Pr\big[\big|\|\mathbf{R}\boldsymbol{x}\|_2^2 - \|\boldsymbol{x}\|_2^2\big| > \delta \|\boldsymbol{x}\|_2^2\big] \leq 2 \exp\big(-T\big(\tfrac{\delta^2}{4} - \tfrac{\delta^3}{6}\big)\big).$$

Applying this bound with $\delta = \epsilon_r$ to the two fixed vectors $\mathbf{p}_G + \mathbf{p}_H$ and $\mathbf{p}_G - \mathbf{p}_H$, and taking a union bound, both norm-preservation events hold except with probability at most $4\exp(-T(\frac{\epsilon_r^2}{4} - \frac{\epsilon_r^3}{6}))$.

Conditioned on these two events, the polarization identity yields

$$\begin{aligned}|\langle \mathbf{R}\mathbf{p}_G, \mathbf{R}\mathbf{p}_H \rangle - \langle \mathbf{p}_G, \mathbf{p}_H \rangle| &\leq \tfrac{\epsilon_r}{4}\big(\|\mathbf{p}_G + \mathbf{p}_H\|_2^2 + \|\mathbf{p}_G - \mathbf{p}_H\|_2^2\big) \\ &= \tfrac{\epsilon_r}{2}\big(\|\mathbf{p}_G\|_2^2 + \|\mathbf{p}_H\|_2^2\big) \leq \epsilon_r.\end{aligned}$$

Finally, since $\epsilon_r \in (0, 1)$, $\frac{\epsilon_r^2}{4} - \frac{\epsilon_r^3}{6} = \epsilon_r^2\big(\frac{1}{4} - \frac{\epsilon_r}{6}\big) \geq \frac{\epsilon_r^2}{12}$. Therefore, $\Pr[|\widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) - \kappa_L(\boldsymbol{v}, \boldsymbol{w})| > \epsilon_r] \leq 4\exp(-\frac{T\epsilon_r^2}{12}) \leq p_f$. □

4.4 Lazy Walk Approximation

While the estimator in Eq. (7) is unbiased, evaluating the exact sketch projections $z_{G,\tau}$ and $z_{H,\tau}$ requires costly estimation for multi-hop transition probabilities. To resolve this bottleneck, we approximate these probabilities using empirical frequencies from lazy random walks. Concretely, we use Algorithm 1, where each call to Lazy-Walk($G, \boldsymbol{v}_G, p, L$) samples a start node $x \sim \boldsymbol{v}_G$, repeatedly either stays at the current node with probability p or moves to a

random neighbor, where we set the lazy factor to $p = 1 - \sqrt{1-\alpha}$ to align with the geometric weights, and stops as soon as the walk decides to stay or reaches the maximal length L. The algorithm returns the endpoint x together with its length ℓ.

We run this procedure W_G times on G, obtaining independent pairs $\{(x_i, L_i)\}_{i=1}^{W_G}$, and similarly W_H lazy walks on H. For graph G, we aggregate the endpoints of all W_G walks into an empirical length-histogram $\widehat{a}_\ell$, defined as the importance-scaled sum of contributions at each length ℓ: $\widehat{a}_\ell = \frac{1}{W_G} \sum_{i=1}^{W_G} \frac{w_{L_i}}{p(1-p)^{L_i}} \boldsymbol{w}_G(x_i)\, \mathbb{I}(L_i, \ell)$. Once all walks terminate, we perform a projection by multiplying this histogram with the Rademacher vectors to build the empirical estimate $\tilde{z}_{G,\tau} = \sum_{\ell=0}^{L} \boldsymbol{g}_\tau(\ell)\, \widehat{a}_\ell$ of the exact projection $z_{G,\tau}$. The fingerprints on H are treated in the same way, yielding $\tilde{z}_{H,\tau}$. Plugging these into the decoupled estimator gives the Monte Carlo version

$$\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) := \frac{1}{T} \sum_{\tau=1}^{T} \tilde{z}_{G,\tau}\, \tilde{z}_{H,\tau}. \tag{8}$$

Lemma 4.5. *For any product-form query vectors $\boldsymbol{v}$ and $\boldsymbol{w}$, the lazy-walk estimator* (8) *satisfies* $\mathbb{E}\big[\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})\big] = \kappa_L(\boldsymbol{v}, \boldsymbol{w})$.

Proof. We first analyze a single sketch index such as $\tau = 1$. As defined in the algorithmic description, the empirical projection is $\tilde{z}_{G,1} = \sum_{\ell=0}^{L} \boldsymbol{g}_1(\ell)\, \widehat{a}_\ell$. The probability that a lazy walk stops at exactly step ℓ is $\mathbb{E}[\mathbb{I}(L_i, \ell)] = p(1-p)^\ell$. Conditioned on stopping at step ℓ, the expected endpoint distribution follows the ℓ-step transition from $\boldsymbol{v}_G$, meaning $\mathbb{E}[\boldsymbol{w}_G(x_i) \mid L_i = \ell] = \boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G$. By the law of total expectation, the expected value of $\widehat{a}_\ell$ is

$$\begin{aligned}\mathbb{E}[\widehat{a}_\ell] &= \tfrac{w_\ell}{p(1-p)^\ell} \cdot \mathbb{E}[\mathbb{I}(L_i, \ell)] \cdot \mathbb{E}[\boldsymbol{w}_G(x_i) \mid L_i = \ell] \\ &= \tfrac{w_\ell}{p(1-p)^\ell} \cdot p(1-p)^\ell \cdot (\boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G) = w_\ell\, \boldsymbol{v}_G^\top \mathbf{P}_G^\ell \boldsymbol{w}_G.\end{aligned}$$

By the linearity of expectation, $\mathbb{E}[\tilde{z}_{G,1}] = \sum_{\ell=0}^{L} \boldsymbol{g}_1(\ell)\, \mathbb{E}[\widehat{a}_\ell] = z_{G,1}$. Conditioned on the $\boldsymbol{g}_1$, the random walks on G and H are independent, and hence $\mathbb{E}_{\text{walk}}[\tilde{z}_{G,1}\tilde{z}_{H,1} \mid \boldsymbol{g}_1] = z_{G,1} z_{H,1}$. Taking expectation over $\boldsymbol{g}_1$ and applying Lemma 4.3 ends the proof. □

Beyond expectation. Below, we provide a concentration bound on how many walks are required to meet a desired accuracy.

Lemma 4.6. *Let $S(\alpha) = \frac{1+\sqrt{1-\alpha}}{\sqrt{\alpha}}$. For any $\epsilon_w > 0$ and $p_f \in (0,1)$, if the number of walks $W_G, W_H = O(S(\alpha)^4 \epsilon_w^{-2} \log \frac{(L+1)T}{p_f})$, then* $\Pr[|\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) - \widehat{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})| > \epsilon_w] \le p_f$.

Proof. For a fixed sketch index τ, substituting the definition of $\widehat{a}_\ell$ into $\tilde{z}_{G,\tau}$ shows that the empirical projection is the average of W_G independent random variables, specifically $\frac{w_{\ell_i}}{p(1-p)^{\ell_i}} \boldsymbol{w}_G(x_i)\, \boldsymbol{g}_\tau(\ell_i)$. Each of these independent terms is bounded by $S(\alpha)$. By Hoeffding's inequality, the deviation probability is $\Pr\big[|\tilde{z}_{G,\tau} - z_{G,\tau}| > \eta\big] \le 2\exp\big(-\Omega\big(W_G \eta^2 / S(\alpha)^2\big)\big)$. To ensure the total error is within ϵ_w, we require the individual deviations to satisfy $\eta = \Theta(\epsilon_w / S(\alpha))$. Substituting this η into the inequality reveals that the exponent scales as $\Omega(W_G \epsilon_w^2 / S(\alpha)^4)$. Applying a union bound over all sketches and both graphs confirms that the sample complexity stated in the lemma restricts the failure probability to at most p_f. □

Cost reduction via reducing the number of walks. Recall that Lemma 4.6 gives a baseline walk budget $W^{(0)}$ sufficient to control the sampling error. However, simulating a large number of random walks can still be practically expensive. We thus compute a short prefix exactly and use random walks only for the remaining tail.

Consider graph G, and let $\ell_s \in \{0, \dots, L\}$ be the prefix length. Starting from the source distribution $\boldsymbol{s} = \boldsymbol{v}_G^\top$, we iteratively update $\boldsymbol{s} \leftarrow \boldsymbol{s}\mathbf{P}_G$. After ℓ_s steps, $\boldsymbol{s} = \boldsymbol{v}_G^\top \mathbf{P}_G^{\ell_s}$. Since $\mathbf{P}_G$ is row-stochastic, $\boldsymbol{s}$ remains a probability distribution. For every residual length $k \ge 0$ with $\ell_s + k \le L$, $\boldsymbol{v}_G^\top \mathbf{P}_G^{\ell_s + k} = \boldsymbol{s}\mathbf{P}_G^k$. Thus, the hop contributions for lengths $0, \dots, \ell_s - 1$ are computed exactly, while the remaining tail is estimated by random walks starting from $\boldsymbol{s}$. The deterministic prefix also reduces the magnitude of each random-walk contribution in the tail. For a residual walk of length k, its importance factor is

$$\frac{w_{\ell_s + k}}{p(1-p)^k} = \frac{\sqrt{\alpha}(1-\alpha)^{(\ell_s+k)/2}}{p(1-p)^k} = S(\alpha)(1-p)^{\ell_s}, \tag{9}$$

where $1 - p = \sqrt{1-\alpha}$ and $S(\alpha) = \sqrt{\alpha}/p$. Since $\boldsymbol{w}_G(x) \in [0,1]$ and the Rademacher signs have unit magnitude, each tail summand is bounded by $S(\alpha)(1-p)^{\ell_s}$, instead of $S(\alpha)$ in the baseline sampler.

The Hoeffding exponent used in Lemma 4.6 scales with the walk budget divided by the squared range of a summand. Hence, to preserve the same exponent, it suffices to use

$$W_{\ell_s} := \big\lceil (1-p)^{2\ell_s} W^{(0)} \big\rceil = \big\lceil (1-\alpha)^{\ell_s} W^{(0)} \big\rceil \tag{10}$$

attempted walks for the residual tail. Thus, a longer exact prefix progressively reduces the number of random walks required for the remaining computation. To choose the prefix length in practice, we greedily compare the cost of one additional exact sparse propagation with a conservative upper bound on the remaining sampling cost. Let $V_s = \{v \in V_G \mid \boldsymbol{s}_v > 0\}$ denote the set of currently active nodes. We switch to random-walk sampling once $\sum_{v \in V_s} \boldsymbol{d}_v \ge W_{\ell_s}(L - \ell_s + 1)$. The left-hand side is the cost of one additional sparse propagation, while the right-hand side upper-bounds the residual walk-sampling work under the remaining horizon.

Lemma 4.7. *For any prefix lengths ℓ_G, ℓ_H, deterministic-prefix switching with the residual walk budgets defined in Eq.* (10) *preserves the sampling-error guarantee of Lemma 4.6.*

Proof. We prove the claim for G; the argument for H is identical. The contributions for lengths $0, \dots, \ell_G - 1$ are computed exactly and therefore introduce no sampling error. After these ℓ_G steps, the tail sampler starts from $\boldsymbol{s}_G = \boldsymbol{v}_G^\top \mathbf{P}_G^{\ell_G}$. For every residual length k, $w_{\ell_G + k} \boldsymbol{s}_G \mathbf{P}_G^k \boldsymbol{w}_G = w_{\ell_G + k} \boldsymbol{v}_G^\top \mathbf{P}_G^{\ell_G + k} \boldsymbol{w}_G$, which is exactly the corresponding coordinate of the original profile. Hence, switching to the prefix does not change the expectation of the tail estimator.

It remains to show that the reduced walk budget preserves the same concentration guarantee. By Eq. (9), each random-walk summand in the tail is bounded by $S(\alpha)(1-p)^{\ell_G}$. Let $W_G^{(0)}$ be a baseline walk budget satisfying Lemma 4.6, and choose $W_G = \lceil (1-p)^{2\ell_G} W_G^{(0)} \rceil$. Then, for the same deviation threshold η, the exponent in Hoeffding's inequality satisfies $\frac{W_G \eta^2}{2S(\alpha)^2 (1-p)^{2\ell_G}} \ge \frac{W_G^{(0)} \eta^2}{2S(\alpha)^2}$. Therefore, reducing the walk budget by the squared factor $(1-p)^{2\ell_G}$ does not increase the deviation probability. The residual lengths form a subset of those considered in Lemma 4.6, while the number of sketches is unchanged, so the same union-bound argument applies. Together with the exact prefix, this preserves the original sampling-error guarantee. □

Algorithm 2: CANWAS

Input: Graphs G, H; vectors $\boldsymbol{v}_G, \boldsymbol{w}_G$ on V_G and $\boldsymbol{v}_H, \boldsymbol{w}_H$ on V_H; truncation depth L; parameter α; number of sketches T; walk budgets W_G, W_H; deterministic prefix lengths ℓ_G, ℓ_H

Output: Estimated kernel $\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})$

$p \leftarrow 1 - \sqrt{1-\alpha}$

Initialize arrays h_G, h_H of size $L+1$ with zeros

for $\tau \leftarrow 1$ **to** T **do**

  Sample a Rademacher vector $\boldsymbol{g}_\tau \in \{\pm 1\}^{L+1}$

  $z_G[\tau] \leftarrow 0,\ z_H[\tau] \leftarrow 0$

// (1) Deterministic traversal prefix on G and H

$\boldsymbol{s}_G \leftarrow \boldsymbol{v}_G^\top$

for $\ell \leftarrow 0$ **to** $\ell_G - 1$ **do** $h_G[\ell] \leftarrow w_\ell \boldsymbol{s}_G \boldsymbol{w}_G$; $\boldsymbol{s}_G \leftarrow \boldsymbol{s}_G \mathbf{P}_G$

Repeat lines 7-11 on graph H analogously to populate h_H

// (2) Lazy-walk tails on G and H

for $w \leftarrow 1$ **to** W_G **do**

  $(x, k, \text{valid}) \leftarrow \textsc{Lazy-Walk}(G, \boldsymbol{s}_G, p, L - \ell_G)$

  if valid = 1 **then**

    $h_G[\ell_G + k] \leftarrow h_G[\ell_G + k] + \frac{1}{W_G} \frac{w_{\ell_G + k}}{p(1-p)^k} \boldsymbol{w}_G(x)$

for $w \leftarrow 1$ **to** W_H **do**

  $(y, k, \text{valid}) \leftarrow \textsc{Lazy-Walk}(H, \boldsymbol{s}_H, p, L - \ell_H)$

  if valid = 1 **then**

    $h_H[\ell_H + k] \leftarrow h_H[\ell_H + k] + \frac{1}{W_H} \frac{w_{\ell_H + k}}{p(1-p)^k} \boldsymbol{w}_H(y)$

for $\ell \leftarrow 0$ **to** L **do**

  for $\tau \leftarrow 1$ **to** T **do**

    $z_G[\tau] \leftarrow z_G[\tau] + h_G[\ell] \cdot \boldsymbol{g}_\tau(\ell)$

    $z_H[\tau] \leftarrow z_H[\tau] + h_H[\ell] \cdot \boldsymbol{g}_\tau(\ell)$

// (3) Final decoupled estimator

$\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) \leftarrow 0$

for $\tau \leftarrow 1$ **to** T **do** $\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) \leftarrow \widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) + z_G[\tau] \cdot z_H[\tau]$

$\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) \leftarrow \widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})/T$

return $\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})$

4.5 Total Complexity Analysis

Based on the step-wise analysis developed in previous sections, we now present our final algorithm CANWAS outlined in Algorithm 2, which offers provable error guarantees with superior efficiency.

At a high level, CANWAS proceeds in three stages. The algorithm initializes two fixed-size empirical histograms, h_G and h_H (each of size $L+1$). First, it performs a short deterministic traversal on G and H. Specifically, starting from the source distributions $\boldsymbol{v}_G$ and $\boldsymbol{v}_H$, it applies a few steps of sparse matrix-vector multiplication with $\mathbf{P}_G$ and $\mathbf{P}_H$ to obtain the exact probability mass of the first ℓ_G and ℓ_H walk lengths, directly populating the initial elements of h_G and h_H. Second, it draws W_G lazy random walks on G and W_H on H, using Algorithm 1 with lazy factor p and truncation depth L. Finally, the algorithm performs a batched projection of these fully populated histograms (h_G and h_H) into the T-dimensional Rademacher sketches. It then combines the two sketches via the decoupled estimator (8) and returns the normalized average over the T trials as the estimated kernel value.

We now state the end-to-end accuracy and complexity guarantee.

Lemma 4.8. *Fix an error $\epsilon > 0$ and failure probability $p_f \in (0,1)$. With parameters configured appropriately, CANWAS runs in time $O\big((\log(1/\epsilon))^2/\epsilon^2\big)$ and requires $O(\epsilon^{-2}\log(1/p_f))$ memory, and returns $\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w})$ satisfying $\Pr[|\widetilde{\kappa}_L(\boldsymbol{v}, \boldsymbol{w}) - \kappa(\boldsymbol{v}, \boldsymbol{w})| \le \epsilon] \ge 1 - p_f$.*

Proof. By the triangle inequality, the total estimation error can be decomposed into truncation error, sketching error, and lazy-walk sampling error $|\widetilde{\kappa}_L - \kappa| \le |\kappa_L - \kappa| + |\widehat{\kappa}_L - \kappa_L| + |\widetilde{\kappa}_L - \widehat{\kappa}_L|$. To ensure the total error is at most ϵ, we assign a uniform budget of $\epsilon/3$ to each term and select the parameters based on Lemmas 4.1, 4.4, and 4.6. Specifically, we set the truncation depth $L = O(\log(1/\epsilon))$ to bound the first term. For the sketching term, we set the sketch size $T = O(\epsilon^{-2}\log(1/p_f))$ to ensure the error is within $\epsilon/3$ with probability at least $1 - p_f/3$. Similarly, for the sampling term, we set the walk counts $W_G, W_H = O(\epsilon^{-2}\log\frac{(L+1)T}{p_f})$, which guarantees the error bound with probability at least $1 - p_f/3$. Applying a union bound, the total error exceeds ϵ with probability at most p_f.

We next analyze the complexity. The sparse multiplications are halted when the cost exceeds the walk simulation cost. Consequently, the deterministic time $\mathcal{T}_{\text{det}}$ is bounded by $O(W_G L + W_H L)$. In the random walk phase, aggregating the endpoints into the local histograms takes $O(1)$ time per walk, bounded by $O(W_G + W_H)$. The final projection multiplies these $(L+1)$-sized histograms with the T Rademacher vectors, taking $O(L \cdot T)$ operations. Thus, the total time is bounded by $\mathcal{T} = O\big(W_G L + W_H L + L \cdot T\big)$. Substituting $L = O(\log(1/\epsilon))$, $T = O(\epsilon^{-2}\log(1/p_f))$, and $W_G, W_H = \tilde{O}(\epsilon^{-2})$—the time complexity simplifies to $O((\log(1/\epsilon))^2/\epsilon^2)$. Finally, the working memory is bounded by the T-dimensional sketches, yielding the stated space complexity. □

5 SINGLE-SOURCE RWK

Beyond global similarity, graph database applications often require *provenance*—point-to-point correspondences that explain a graph-level match. In seeded graph matching [13, 18], a trusted pair (s, t), e.g., corresponding landmarks in point-cloud matching [41], provides such an anchor. A single-source query propagates its structural evidence to all candidate pairs $(u, v) \in V_\times$ by evaluating $\kappa_{s,t}(u, v) = \kappa(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})$; high-scoring pairs highlight local correspondences supporting the overall similarity. Since evaluating every target independently is prohibitive, we construct reusable node fingerprints that allow us to decode any target score from a single construction while preserving rigorous error guarantees.

5.1 Lifting the Analysis

Extending the ideas in Section 4 to the single-source case requires addressing two key challenges: (i) guaranteeing *simultaneous* accuracy over $|V_\times|$ entries, and (ii) avoiding the $O(|V_\times|)$ storage required to materialize the full kernel vector.

Our first idea for addressing the challenges is based on series truncation, which carries over readily. Truncating the geometric walk series after $L = \lceil \log_{1-\alpha} \epsilon_t \rceil - 1$ ensures that the residual tail contributes at most ϵ_t additive error to *every* entry uniformly. Our second idea is aimed at ensuring simultaneous accuracy across the entire product graph, which requires a tighter sketch bound. Applying Lemma 4.4 to every target pair and taking a union bound over $V_\times$, it suffices to choose $T \ge 12\epsilon_r^{-2}\log(\frac{8|V_\times|}{p_f})$ to ensure that the projection error is at most ϵ_r simultaneously for all targets.

Random walk sampling for global accuracy. In the single-source setting, the estimator must control the transition histograms from s and t to all targets, i.e., all entries $(\ell, u) \in [0, L] \times V_G$ and $(\ell, v) \in [0, L] \times V_H$, rather than a single scalar kernel value.

We simulate W_G and W_H lazy walks from node s and node t respectively, and aggregate them into empirical transition distributions. Let $q_G^k(s, u)$ denote the empirical frequency that a walk from s stops on u at step k. To avoid storing dense quadratic pairs, we construct compressed fingerprints for every node. For each Rademacher vector $\boldsymbol{g}_\tau \in \{\pm 1\}^{L+1}$, we compute

$$\mathrm{vec}_G^\tau[u] := \sum_{k=0}^{L} \boldsymbol{g}_\tau(k) \cdot q_G^k(s, u),\ \mathrm{vec}_H^\tau[v] := \sum_{k=0}^{L} \boldsymbol{g}_\tau(k) \cdot q_H^k(t, v).$$

The kernel is estimated via the inner product of these fingerprints

$$\widetilde{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v}) := \tfrac{\alpha}{(1-\sqrt{1-\alpha})^2} \cdot \frac{1}{T} \sum_{\tau=1}^{T} \mathrm{vec}_G^\tau[u] \cdot \mathrm{vec}_H^\tau[v].$$

To analyze the approximation error introduced by these empirical estimates, we first ensure that the walk histograms themselves are accurate uniformly.

Lemma 5.1. *Suppose the number of random walks satisfies*

$$W_G \geq \tfrac{2S(\alpha)^4(L+1)^2}{\epsilon_w^2} \log((4(L+1)|V_G|)/p_f),$$

$$W_H \geq \tfrac{2S(\alpha)^4(L+1)^2}{\epsilon_w^2} \log((4(L+1)|V_H|)/p_f),$$

where $S(\alpha) = \frac{1+\sqrt{1-\alpha}}{\sqrt{\alpha}}$. *Then* $\max_{\ell,u} |q_G^\ell(s, u) - p(1-p)^\ell p_G^\ell(s, u)| \leq \frac{\epsilon_w}{2S(\alpha)(L+1)}$ *and* $\max_{\ell,v} |q_H^\ell(t, v) - p(1-p)^\ell p_H^\ell(t, v)| \leq \frac{\epsilon_w}{2S(\alpha)(L+1)}$ *hold simultaneously with probability at least* $1 - p_f/4$.

Proof. For a fixed step $\ell \in [0, L]$ and node $u \in V_G$, the empirical frequency $q_G^\ell(s, u)$ is the mean of W_G independent Bernoulli random variables with expectation $p(1-p)^\ell p_G^\ell(s, u)$. Let the deviation be $\delta = \frac{\epsilon_w}{2S(\alpha)^2(L+1)}$. By Hoeffding's inequality, the probability that the estimate exceeds this deviation is bounded by $2\exp(-2W_G\delta^2)$.

Substituting the given sample complexity for W_G and noting that $S(\alpha) \geq 1$, the failure probability for a single bin is bounded by $2\exp(-S(\alpha)^2 \log(\frac{4(L+1)|V_G|}{p_f})) \leq \frac{p_f}{4(L+1)|V_G|}$. Applying a union bound over all $(L+1)|V_G|$ bins in the walk-histogram, the global error guarantee holds for G with probability at least $1 - p_f/4$. An identical symmetric argument ensures the bound holds for H. □

Next, we bound the error propagated from these approximate histograms to the final estimator, relative to the exact sketch $\widehat{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})$.

Lemma 5.2. *Based on Lemma 5.1, with probability at least* $1 - p_f/2$, *it holds for all* $(u, v) \in V_\times$ *that* $|\widetilde{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v}) - \widehat{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})| \leq \epsilon_w$. *For fixed* α, *by setting* $n_m = \max\{n_G, n_H\}$, *the algorithm runs in*

$$O(S(\alpha)^4\epsilon^{-2}\log^3(1/\epsilon)\log\frac{n_m\log(1/\epsilon)}{p_f} + (n_G+n_H)\epsilon^{-2}\log(1/\epsilon)\log\frac{n_\times}{p_f})$$

time, and requires $O((n_G + n_H)[\log(1/\epsilon) + \epsilon^{-2}\log\frac{n_\times}{p_f}])$ *space.*

Proof. Let $r_G^\ell(s, u) = p(1-p)^\ell p_G^\ell(s, u)$ and $r_H^\ell(t, v) = p(1-p)^\ell p_H^\ell(t, v)$. Define the corresponding exact fingerprints as $\widehat{\mathrm{vec}}_G^\tau[u] = \sum_{\ell=0}^{L} \boldsymbol{g}_\tau(\ell) r_G^\ell(s, u)$, $\widehat{\mathrm{vec}}_H^\tau[v] = \sum_{\ell=0}^{L} \boldsymbol{g}_\tau(\ell) r_H^\ell(t, v)$. By Lemma 5.1, simultaneously for every node and sketch, $|\mathrm{vec}_G^\tau[u] - \widehat{\mathrm{vec}}_G^\tau[u]| \leq \frac{\epsilon_w}{2S(\alpha)^2}$, and the same bound holds for H.

Moreover, since each attempted walk contributes to at most one stopping-length bin, we have $|\mathrm{vec}_G^\tau[u]| \leq \sum_{\ell=0}^{L} q_G^\ell(s, u) \leq 1$, while $|\widehat{\mathrm{vec}}_G^\tau[u]| \leq \sum_{\ell=0}^{L} r_G^\ell(s, u) \leq 1$. The same bounds hold for graph H.

Using $ab - \hat{a}\hat{b} = (a - \hat{a})b + \hat{a}(b - \hat{b})$, the error of each fingerprint product is therefore at most $\epsilon_w/S(\alpha)^2$. Since $\alpha/p^2 = S(\alpha)^2$, averaging over the T sketches gives $\left|\widetilde{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v}) - \widehat{\kappa}_L(\boldsymbol{e}_{s,t}, \boldsymbol{e}_{u,v})\right| \leq \epsilon_w$. Since the event in Lemma 5.1 holds simultaneously for all histogram bins, the above bound also holds simultaneously for all target pairs with probability at least $1 - p_f/2$.

For complexity, each random-walk trial performs at most L transitions, so constructing the empirical histograms takes $O((W_G + W_H)L)$ time. Constructing the T-dimensional fingerprints for all nodes takes $O((n_G + n_H)LT)$ time. By Lemma 5.1, under the uniform error allocation, $W_G, W_H = O(S(\alpha)^4\epsilon^{-2}L^2\log\frac{Ln_m}{p_f})$. Moreover, $L = O(\log(1/\epsilon))$ and $T = O(\epsilon^{-2}\log\frac{n_\times}{p_f})$. Substituting these bounds yields the stated running time.

Finally, storing the histograms and the T-dimensional fingerprints requires $O((n_G+n_H)(L+T))$ space, assuming the Rademacher vectors are generated on demand. Substituting the bounds on L and T yields the stated space complexity. □

6 EXTENSION TO NODE-LABELED GRAPHS

The preceding sections consider RWK queries on unlabeled graphs. We now extend CANWAS to discrete node labels, where matching walks must have both the same length and the same label at every corresponding position, including their start and endpoint. This condition filters walk-pair contributions without changing or renormalizing the original transition probabilities.

A length histogram merges walks with different label sequences and therefore loses the information needed for this matching condition. Retaining a separate bin for every observed sequence would resolve the ambiguity, but introduces pattern-dependent storage and projection work. We instead extend the shared Rademacher representation to complete walk signatures and construct it directly from sampled walks. Below, we reuse the truncation and projection analysis and establish the additional sampling and resource bounds for this extension.

6.1 Label-Aware Projections

Let $\lambda_X(u) \in \Sigma$ denote the label of node u in graph $X \in \{G, H\}$, with a common finite alphabet Σ. We consider fixed product-form queries $\boldsymbol{v} = \boldsymbol{v}_G \otimes \boldsymbol{v}_H$ and $\boldsymbol{w} = \boldsymbol{w}_G \otimes \boldsymbol{w}_H$, where $\boldsymbol{v}_X$ is a probability distribution and $\boldsymbol{w}_X \in [0, 1]^{n_X}$. The graphs, labels, and query vectors are independent of the algorithm's randomness. Recall $w_\ell = \sqrt{\alpha}(1-\alpha)^{\ell/2}$, $p = 1 - \sqrt{1-\alpha}$, and $S(\alpha) = \sqrt{\alpha}/p$.

A walk signature $(\ell, \mathbf{c})$ specifies its length and visited label sequence $\mathbf{c} \in \Sigma^{\ell+1}$. Let $a_X(\ell, \mathbf{c})$ be the total $\boldsymbol{w}_X$-weighted probability mass of length-ℓ walks from $\boldsymbol{v}_X$ with sequence $\mathbf{c}$. Partitioning walk pairs by their common sequence and using independence between the two graph-side walks gives

$$\kappa_L^{\mathrm{lab}}(\boldsymbol{v}, \boldsymbol{w}) = \sum_{\ell=0}^{L} \sum_{\mathbf{c} \in \Sigma^{\ell+1}} w_\ell^2 a_G(\ell, \mathbf{c}) a_H(\ell, \mathbf{c}). \tag{11}$$

The infinite sum defines κ^{lab}; assigning all nodes a common label recovers the unlabeled kernel.

For analysis, define the signature profile $\mathbf{p}_X[(\ell, \mathbf{c})] = w_\ell a_X(\ell, \mathbf{c})$ over $\mathcal{K}_L = \bigcup_{\ell=0}^{L} \{\ell\} \times \Sigma^{\ell+1}$. The sequence events partition the ordinary length-ℓ walks, so $\sum_{\mathbf{c}} a_X(\ell, \mathbf{c}) = \boldsymbol{v}_X^\top \mathbf{P}_X^\ell \boldsymbol{w}_X \leq 1$. Since these

masses are nonnegative,

$$\|\mathbf{p}_X\|_1 \le \sum_{\ell=0}^{L} w_\ell \le S(\alpha),$$
$$\|\mathbf{p}_X\|_2^2 \le \sum_{\ell=0}^{L} w_\ell^2 \left(\sum_{\mathbf{c}} a_X(\ell,\mathbf{c})\right)^2 \le 1. \tag{12}$$

Each length's unweighted contribution is also at most one. The geometric-tail argument of Lemma 4.1 therefore gives $0 \le \kappa^{\mathrm{lab}} - \kappa_L^{\mathrm{lab}} \le (1-\alpha)^{L+1}$, including every single-source target.

Following the shared-randomness principle of [15], replace $g_\tau(\ell)$ by $g_\tau(\ell,\mathbf{c})$. We first assume mutually independent Rademacher signs across distinct $(\tau,\ell,\mathbf{c})$ indices, independent of the walks and shared between both graph roles. Their implementation costs are addressed below. Define

$$z_{X,\tau} = \sum_{\ell=0}^{L} \sum_{\mathbf{c}\in\Sigma^{\ell+1}} w_\ell a_X(\ell,\mathbf{c}) g_\tau(\ell,\mathbf{c}),$$
$$\widehat{\kappa}_L^{\mathrm{lab}} = T^{-1} \sum_{\tau=1}^{T} z_{G,\tau} z_{H,\tau}. \tag{13}$$

The sign-correlation identity retains exactly the equal-signature terms, so $\mathbb{E}[\widehat{\kappa}_L^{\mathrm{lab}}] = \langle \mathbf{p}_G, \mathbf{p}_H \rangle = \kappa_L^{\mathrm{lab}}$. By Eq. (12), the dimension-independent norm-preservation argument in Lemma 4.4 applies unchanged. Consequently, for $\epsilon_r, p_r \in (0,1)$,

$$T \ge 12\epsilon_r^{-2} \log(4/p_r) \tag{14}$$

bounds the sketch error by ϵ_r except with probability p_r. Neither the profiles nor their index set need to be materialized.

6.2 Block-Wise Estimation

To avoid projecting every observed signature into all T coordinates, we divide W_X independent attempted walks into T blocks of $B_X = W_X/T$ trials. Each block updates one sketch, initially zero; $W_X \ge T$ is rounded up to a multiple of T. A trial uses Algorithm 1 with source $\boldsymbol{\nu}_X$, probability p, and horizon L, additionally recording all visited labels. As in that algorithm, a failed stopping trial at length L returns an invalid sample. Thus $\Pr(J=\ell, \mathrm{valid}=1) = p(1-p)^\ell$ for $0 \le \ell \le L$. A valid trial in block τ, ending at x with signature $(J,\mathbf{c})$, performs

$$\widetilde{z}_{X,\tau} \mathrel{+}= \frac{S(\alpha)}{B_X} \boldsymbol{w}_X(x) g_\tau(J,\mathbf{c}). \tag{15}$$

Invalid trials contribute zero but remain included in B_X. Using independent walk trials across blocks and graph roles, we return $\widetilde{\kappa}_L^{\mathrm{lab}} = T^{-1}\sum_{\tau=1}^{T} \widetilde{z}_{G,\tau}\widetilde{z}_{H,\tau}$. The ordinary deterministic prefix is disabled because its propagated state retains endpoint mass but not the required label history.

Unlike the main-text histogram construction, different sketch coordinates now use disjoint walk batches. We therefore analyze the sampling error of this estimator directly.

Lemma 6.1. *Conditioned on the shared signs g, $\mathbb{E}_{\mathrm{walk}}[\widetilde{\kappa}_L^{\mathrm{lab}} \mid g] = \widehat{\kappa}_L^{\mathrm{lab}}$. For $\epsilon_w > 0$ and $p_w \in (0,1)$, divisible walk budgets satisfying*

$$W_X \ge \max\{T, 8S(\alpha)^4 \epsilon_w^{-2} \log(4/p_w)\} \tag{16}$$

ensure $\Pr(|\widetilde{\kappa}_L^{\mathrm{lab}} - \widehat{\kappa}_L^{\mathrm{lab}}| > \epsilon_w) \le p_w$.

Proof. Let $Y_{X,\tau,i}$ be the numerator of the i-th update in Eq. (15), or zero for an invalid trial. Then $\widetilde{z}_{X,\tau} = B_X^{-1}\sum_i Y_{X,\tau,i}$ and we have $|Y_{X,\tau,i}|, |\widetilde{z}_{X,\tau}|, |z_{X,\tau}| \le S(\alpha)$. Conditioned on a valid stop at length ℓ, the path has the ordinary length-ℓ walk distribution. Since $p(1-p)^\ell S(\alpha) = w_\ell$,

$$\mathbb{E}[Y_{X,\tau,i} \mid g] = \sum_{\ell=0}^{L}\sum_{\mathbf{c}} w_\ell a_X(\ell,\mathbf{c}) g_\tau(\ell,\mathbf{c}) = z_{X,\tau}.$$

Independence of the two graph-role batches proves conditional unbiasedness.

Set $A = T^{-1}\sum_\tau z_{G,\tau}\widetilde{z}_{H,\tau}$. Conditioning further on the H-side walks, write

$$\widetilde{\kappa}_L^{\mathrm{lab}} = \sum_{\tau=1}^{T}\sum_{i=1}^{B_G} \frac{\widetilde{z}_{H,\tau}}{W_G} Y_{G,\tau,i}.$$

These W_G summands are independent, their sum has conditional mean A, and each has interval length at most $2S(\alpha)^2/W_G$. Hoeffding's inequality [28] therefore bounds the probability of $|\widetilde{\kappa}_L^{\mathrm{lab}} - A| > \epsilon_w/2$ by $2\exp[-W_G\epsilon_w^2/(8S(\alpha)^4)]$. This bound remains valid after averaging over the H-side walks. Similarly, conditioned only on g, $A = \sum_{\tau,i} z_{G,\tau}Y_{H,\tau,i}/W_H$ has mean $\widehat{\kappa}_L^{\mathrm{lab}}$ and independent summands of interval length at most $2S(\alpha)^2/W_H$. The second deviation, $|A - \widehat{\kappa}_L^{\mathrm{lab}}| > \epsilon_w/2$, is bounded by the same expression with W_H. The triangle inequality and a union bound give total conditional failure probability at most $2e^{-W_G\epsilon_w^2/(8S(\alpha)^4)} + 2e^{-W_H\epsilon_w^2/(8S(\alpha)^4)} \le p_w$. Averaging over g proves the result. □

Theorem 6.2 (Distributional guarantee). *Let $\epsilon, p_f \in (0,1)$. Choose positive budgets with $\epsilon_t + \epsilon_r + \epsilon_w \le \epsilon$ and $p_r + p_w \le p_f$. Set $L = \lceil \log(1/\epsilon_t)/[-\log(1-\alpha)]\rceil - 1$, choose T by Eq. (14), and use the smallest multiples of T satisfying Eq. (16). Then $\widetilde{\kappa}_L^{\mathrm{lab}}$ is unbiased for κ_L^{lab} and $\Pr(|\widetilde{\kappa}_L^{\mathrm{lab}} - \kappa^{\mathrm{lab}}| \le \epsilon) \ge 1 - p_f$.*

Proof. The tail bound is deterministic; the projection and sampling errors are at most ϵ_r and ϵ_w outside events of probabilities p_r and p_w. Their sum bounds the total error, so a union bound proves accuracy. Taking expectation over the signs in Lemma 6.1 proves unbiasedness for the truncated kernel. □

6.3 Simultaneous Single-Source Queries

To support all targets from a fixed source pair (s,t), we accumulate walk contributions at their endpoints rather than sampling separately for each target. Set $s_G = s$, $s_H = t$, and let $q_X^\ell(s_X,u;\mathbf{c})$ be the probability that an ordinary length-ℓ walk from s_X ends at u with label sequence $\mathbf{c}$. The exact node profile is $\mathbf{p}_{X,u}[(\ell,\mathbf{c})] = w_\ell q_X^\ell(s_X,u;\mathbf{c})$. Its norms satisfy Eq. (12) because $\sum_{\mathbf{c}} q_X^\ell(s_X,u;\mathbf{c}) = \mathbf{p}_X^\ell(s_X,u) \le 1$, and $\kappa_{s,t,L}^{\mathrm{lab}}(u,v) = \langle \mathbf{p}_{G,u}, \mathbf{p}_{H,v}\rangle$.

We use the same block allocation and shared signs, initialize an $n_X \times T$ node-fingerprint table to zero, and start walks from s_X. A valid trial in block τ updates only its endpoint entry: $\widetilde{\mathrm{vec}}_{X,\tau}[x] \mathrel{+}= S(\alpha)g_\tau(J,\mathbf{c})/B_X$. Any target pair is decoded as

$$\widetilde{\kappa}_{s,t,L}^{\mathrm{lab}}(u,v) = \frac{1}{T}\sum_{\tau=1}^{T} \widetilde{\mathrm{vec}}_{G,\tau}[u]\widetilde{\mathrm{vec}}_{H,\tau}[v]. \tag{17}$$

No additional normalization is needed because both updates already include $S(\alpha)$. If $\lambda_G(s) \ne \lambda_H(t)$, all target values are zero and sampling is unnecessary.

Corollary 6.3. *Let $n_\times = n_G n_H$. Use the error allocations and truncation depth of Theorem 6.2, but replace p_r by $p_r/n_\times$ in Eq.* (14) *and p_w by $p_w/n_\times$ in Eq.* (16). *Then*

$$\Pr(\max_{(u,v)\in V_\times} |\widehat{\kappa}^{\mathrm{lab}}_{s,t,L}(u,v) - \kappa^{\mathrm{lab}}_{s,t}(u,v)| \le \epsilon) \ge 1 - p_f.$$

Proof. For a fixed target pair, take $\boldsymbol{w}_G = \boldsymbol{e}_u$ and $\boldsymbol{w}_H = \boldsymbol{e}_v$. Its node-sketch entries are exactly the block estimates of that indicator-weighted query. The profile bounds, conditional unbiasedness, and both Hoeffding steps above therefore apply, giving projection and sampling failure probabilities at most $p_r/n_\times$ and $p_w/n_\times$. Union bounds over all pairs control the two error sources simultaneously; no independence between target estimates is required. The deterministic tail bound then gives the stated total error. □

6.4 Complexity and Representation Reuse

The block-wise construction processes each sampled walk directly, avoiding both a signature histogram and a separate projection pass. We assume constant-time neighbor sampling, label and target-weight lookup, and source sampling from supplied access structures. A consistent random-sign oracle provides the shared independent Rademacher sign for a length-ℓ signature in $O(\ell + 1)$ time. The bounds below exclude the input graphs, supplied access structures, and the oracle's internal state. Sketch arithmetic is counted at unit cost, and accuracy is analyzed in exact arithmetic.

Each attempted walk performs at most L transitions and $L + 1$ stopping tests, records at most $L + 1$ labels, and updates at most one sketch entry. Its processing cost, including sign access, is therefore $O(L + 1)$. Initializing the graph-side sketches takes $O(T)$ time, whereas initializing the node-fingerprint tables takes $O((n_G + n_H)T)$ time. Consequently, distributional construction requires $O((W_G + W_H)(L + 1) + T)$ time, and single-source construction requires $O((W_G + W_H)(L + 1) + (n_G + n_H)T)$ time.

A sequential implementation retains only the current label sequence in addition to the sketch accumulators. The working space is therefore $O(T+L+1)$ for distributional queries and $O((n_G+n_H)T+L+1)$ for single-source queries. After construction, the reusable representations contain $2T$ and $(n_G + n_H)T$ numerical values, respectively. Combining two distributional sketches or decoding one target pair requires $O(T)$ arithmetic operations.

For equal error allocation $\epsilon_t = \epsilon_r = \epsilon_w = \epsilon/3$ and $p_r = p_w = p_f/2$, choose the smallest sufficient integer budgets established above. Let $h_\epsilon = 1 + \log(3/\epsilon)/[-\log(1-\alpha)]$, so that $L + 1 \le h_\epsilon$. Substituting the sketch and walk budgets gives the distributional construction time

$$O(S(\alpha)^4 \epsilon^{-2} h_\epsilon \log \frac{8}{p_f}), \tag{18}$$

with $O(\epsilon^{-2}\log(8/p_f))$ numerical sketch values. For single-source queries, construction takes

$$O(\epsilon^{-2}[S(\alpha)^4 h_\epsilon + n_G + n_H] \log \frac{8n_\times}{p_f}) \tag{19}$$

time and produces $O((n_G+n_H)\epsilon^{-2}\log(8n_\times/p_f))$ numerical sketch values. Decoding one target pair takes $O(\epsilon^{-2}\log(8n_\times/p_f))$ time. During construction, the current label sequence adds $O(h_\epsilon)$ working space beyond the stored sketches. Building source samplers incurs additional work when they are not supplied.

For fixed α, Eqs. (18) and (19) simplify to $O(\epsilon^{-2}\log(3/\epsilon)\log(8/p_f))$ and $O(\epsilon^{-2}(n_G + n_H + \log(3/\epsilon))\log(8n_\times/p_f))$, respectively. These bounds concern the labeled block-wise construction under the stated access model; no enumeration of the signature space is required.

Representations can be reused when their label encoding, truncation depth, sketch dimension, and shared signs agree. Walk batches for the two graph roles must remain independent, including self-comparisons, where two independent copies avoid a squared-noise bias. Explicitly decoding all $n_\times$ target pairs adds $O(n_\times T)$ time and requires $\Theta(n_\times)$ additional space if the scores are stored rather than streamed. The guarantees provide additive accuracy, not relative-error or ranking guarantees. The unlabeled histogram-first algorithm remains unchanged; its finite-sample variance and experimental performance are not inferred from this extension.

7 RELATED WORK

Graph kernel queries have emerged as a fundamental primitive for measuring graph similarity, with applications spanning bioinformatics, social-network analytics, and graph anomaly detection [27, 36, 59, 62]. Existing approaches can be broadly categorized into three classes. (i) *Random-walk and path-based kernels* enumerate label sequences induced by random walks or shortest paths [6, 8, 21, 22, 36, 69]. (ii) *Small-subgraph kernels* represent graphs via the frequency distribution of small induced motifs [29, 39, 63]. (iii) *Subtree-based kernels*, most notably the Weisfeiler–Lehman family, iteratively relabel neighborhood trees and compare the resulting multiset representations [27, 51, 62]. Among these, random-walk kernels (RWKs) are regarded as particularly expressive [9, 61, 69].

A further advantage of RWKs is their flexibility: users can specify customized source distributions and target weights, enabling similarity evaluation under diverse query semantics. This formulation underpins several graph systems. In bioinformatics, it forms the basis of IsoRank [65] for detecting functional orthologs in protein–protein interaction networks. In recommender systems, it supports cross-modal ranking models [42, 56] by identifying user–item pairs that are close in both social and semantic spaces. More recently, RWK-inspired diffusion mechanisms have been incorporated into graph learning architectures: various GNN variants (e.g., [16, 17, 53, 54]) characterize the equilibrium behavior of infinite-depth propagation through random-walk formulations. Recent studies on substructure counting and RWK-based models further suggest that walk-based structural signals remain useful beyond standard message passing [10, 12, 14, 30, 34, 80, 81].

Beyond global similarity modeling, RWKs also enable fine-grained *provenance* analysis by restricting the source distribution to a specific seed pair. In this single-source setting, the kernel produces a contribution map that decomposes overall structural alignment into localized walk-based evidence. This mechanism has been applied in biological sequence alignment, where geometric RWKs score candidate protein correspondences [**?**] and model context-sensitive simultaneous walks [32]. We address the scalability limitation by presenting the first RWK algorithm with near-linear time and memory complexity under rigorous error guarantees, enabling scalable deployment in large graph workloads.

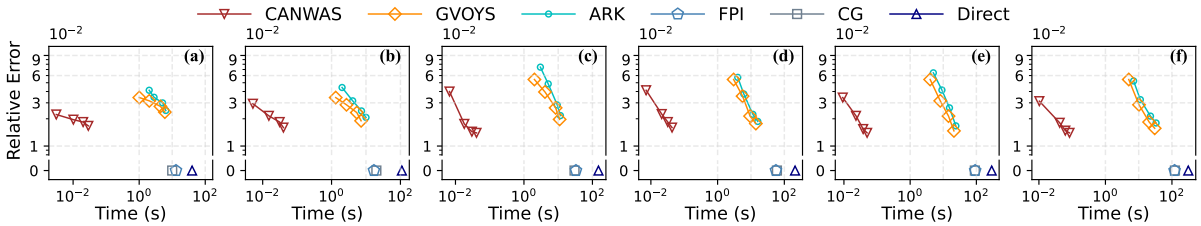


Figure 2: Trade-off between error and running time for RWK query across different algorithms on several graphs:(a) MUTAG, (b) ENZYMES, (c) DBLP, (d) COLLAB,(e) Github, and (f) DD.

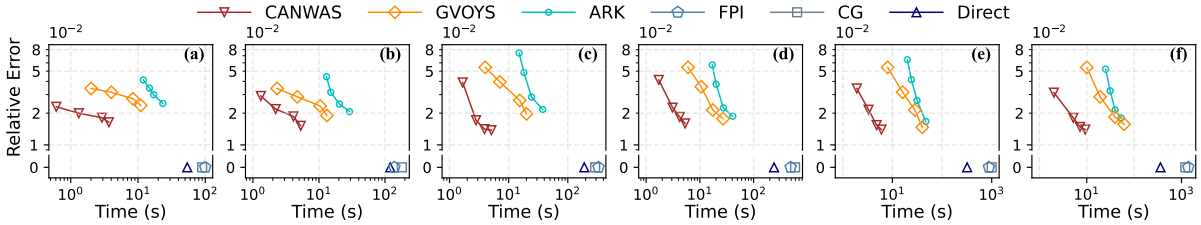


Figure 3: Trade-off between error and running time for single-source RWK query across different algorithms on several graphs: (a) MUTAG, (b) ENZYMES, (c) DBLP, (d) COLLAB, (e) Github, and (f) DD.

Table 2: Statistics of the studied real-world networks.

Network	#nodes	#edges	Size	Network	#nodes	#edges	Size
MUTAG	18	20	Small	DD	284	715	Small
ENZYMES	32	62	Small	Facebook	4,039	88,234	Large
DBLP	52	99	Small	GrQc	4,158	13,428	Large
COLLAB	74	2,457	Small	YouTube	1,134,890	2,987,624	Large
GitHub	113	234	Small	Orkut	3,072,441	117,185,083	Large

Another line of research that is germane to this work concerns efficient computation of random-walk-based graph quantities, including random-walk centrality [46], biharmonic distance [47, 48] and personalized PageRank (PPR) [25, 31, 45]. Among these, PPR is particularly relevant to our algorithmic design and has been extensively studied in the literature, as surveyed in [78]. A number of recent studies [5, 33, 43, 50, 64, 70, 71, 73, 75] propose to combine both push algorithms [2, 3] and random walk sampling [19, 67] for optimizing efficiency in processing PPR queries. Such hybrid push–walk designs provide useful algorithmic context for our greedy deterministic-prefix strategy [43, 79]. However, extending these PPR techniques to our graph-kernel queries is far from straightforward. First, PPR operates on a single graph whereas RWK is naturally defined on the product graph; naïvely running any PPR routine on that product graph would incur quadratic–cubic costs. Second, our method deliberately avoids that explosion by "decoupling" the computation: we approximate, on G and H separately, the hop-by-hop transition vectors and then "recouple" them with variance-controlled Rademacher sketches to recover the kernel value. Classic PPR pipelines cannot deliver a guarantee in the cross-graph setting without major re-design.

8 EXPERIMENTS

8.1 Experimental setting

Datasets and Queries. We assess CANWAS using two dataset regimes. First, we utilize six small collections from TUDataset [52]. Table 2 reports the average vertex and edge counts of the individual graphs in each collection. For evaluation on the small graphs, we randomly sample graph pairs within each TUDataset collection and construct the corresponding direct product graphs for kernel evaluation. Second, we employ four large real-world networks from SNAP [40] (Facebook, GrQc, YouTube, Orkut) to generate six product combinations: Facebook⊗Facebook, Facebook⊗YouTube, Facebook⊗Orkut, GrQc⊗GrQc, GrQc⊗YouTube and GrQc⊗Orkut. For distributional RWK queries, for each graph pair we sample 1,000 query pairs $(\boldsymbol{v}, \boldsymbol{w})$. Each source vector $\boldsymbol{v}$ is a normalized random vector, while each target vector $\boldsymbol{w}$ is a binary vector with entries sampled from $\{0, 1\}$. We report the average error and runtime over these 1k queries. On large product graphs, we issue 10k single-source queries rooted at uniformly sampled individual nodes.

Implementation Details. All experiments were run on a Linux machine with an Intel Xeon(R) Gold 6240@2.60GHz 32-core processor and 512GB of RAM. Before timing, the graph data is loaded into memory. All algorithms are implemented in C++ for a fair comparison. Results are excluded if the algorithm does not return within 24 hours. We set $\alpha = 0.15$ and $p_f = 0.01$ by default. Our code is publicly available at https://github.com/changanliu/graphKernel.

Competitors and Ground Truth. To demonstrate the superiority CANWAS, we compare it with the SOTA algorithms *Ark* and *GVOYS*. Additionally, we consider the exact baselines including the *Direct*, *CG* and *FPI*. For small-scale graphs, we compute the ground truth using Direct. However, for larger-scale graphs, obtaining the exact RWK kernel values using existing methods is computationally infeasible. We therefore compute reference values exactly by sparse power iteration on each factor graph with $L = 1000$, which bounds the error below 10^{-7}.

Evaluation Metrics. Our theoretical analysis establishes rigorous absolute error guarantees. However, because ground-truth kernel values in real-world graphs are typically minuscule, practical applications demand an even stricter standard. To subject our method to this practical test, we evaluate accuracy using the mean relative error over N queries: $\frac{1}{N}\sum_{i=1}^{N}|\frac{\kappa(\boldsymbol{v}_i,\boldsymbol{w}_i)-\widetilde{\kappa}(\boldsymbol{v}_i,\boldsymbol{w}_i)}{\kappa(\boldsymbol{v}_i,\boldsymbol{w}_i)}|$. Empirically, CANWAS consistently achieves exceptionally low relative errors,

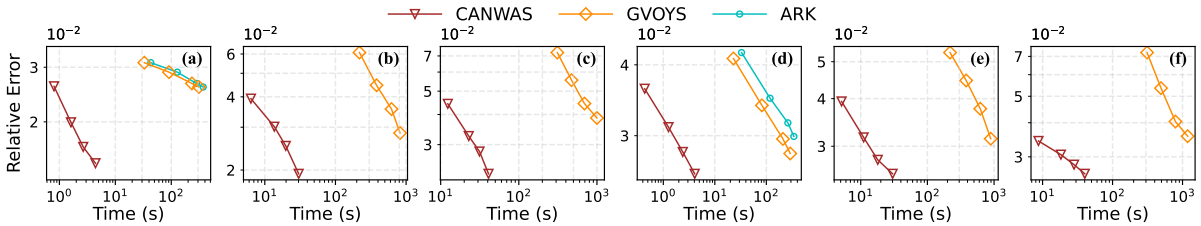


Figure 4: Trade-off between error and running time for RWK query across different algorithms on several graphs: (a) Facebook⊗Facebook, (b) Facebook⊗Youtube, (c) Facebook⊗Orkut, (d) GrQc⊗GrQc, (e) GrQc⊗Youtube, and (f) GrQc⊗Orkut.

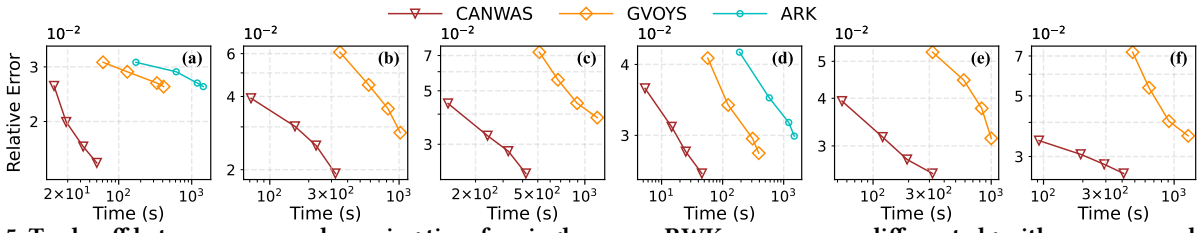


Figure 5: Trade-off between error and running time for single-source RWK query across different algorithms on several graphs: (a) Facebook⊗Facebook, (b) Facebook⊗Youtube, (c) Facebook⊗Orkut, (d) GrQc⊗GrQc, (e) GrQc⊗Youtube, and (f) GrQc⊗Orkut.

proving its robustness beyond the absolute bounds. We report the wall-clock running time in seconds to evaluate efficiency.

8.2 Query Efficiency and Accuracy trade-off

We examine the runtime-versus-accuracy trade-off of RWK evaluators for distributional kernel estimation and single-source queries across two size regimes. For CANWAS, we trace the trade-off curve by varying the target error bound $\epsilon \in \{0.1, 0.05, 0.02, 0.01\}$. Following the sampling-budget scaling in Section 4, the number of Rademacher sketches is $T = \{1\text{k}, 4\text{k}, 25\text{k}, 100\text{k}\}$. The lazy-walk budget W is counted per input graph; we use $W = \{100, 400, 2.5\text{k}, 10\text{k}\}$ on small graphs and $W = \{10\text{k}, 40\text{k}, 250\text{k}, 1\text{M}\}$ on large scale graphs. To ensure a fair comparison, we expand the hyperparameter grids of the baselines until their error rates match those of CANWAS. Specifically, on small graphs we evaluate Ark with feature ranks of 2, 4, 8, and 16, and configure GVOYS with n anchor nodes, a 1k feature dimension, and random walk counts of 2k, 5k, 8k, and 10k. For large graphs, we increase the Ark ranks to 64, 128, 256, and 512, and set the GVOYS parameters to $\sqrt{n}$ anchor nodes, a 10k feature dimension, and walk counts of 0.5, 1, 2, and 4 million, while driving CG and FPI to full numerical convergence.

Figure 2 summarizes the performance on small graphs where CANWAS consistently occupies the optimal lower-left corner. For distributional kernel estimation on MUTAG, our method attains a three-percent error in roughly 0.003s, whereas GVOYS requires nearly two seconds and Ark takes around five. This efficiency pattern holds across ENZYMES, DBLP, COLLAB, and Github, and notably on the densest graph (DD) where our curve remains below one second while competitors lag by at least an order of magnitude. This advantage widens further in the single-source setting (Figure 3) as CANWAS completes within ten seconds on all small graphs, while baselines fall into the multi-minute range.

Figures 4 and 5 extend the comparison to large product graphs. For distributional kernel estimation, CANWAS processes the Facebook and GrQc products in roughly 4s while maintaining error

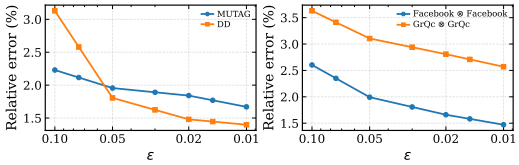


Figure 6: Mean relative error versus ϵ.

rates below four percent. In contrast, GVOYS requires between one and fifteen minutes to achieve comparable accuracy. We note that Ark is limited to the two homogeneous products, taking around 90s on Facebook⊗Facebook, and suffers from memory exhaustion on the cross-network products. In the single-source setting, CANWAS completes all six large graphs within three minutes. Conversely, GVOYS lags at six to twenty minutes with higher error rates, and Ark fails to complete four of the six graphs. Additionally, CG and FPI fail because the product graphs exceed available RAM.

We also evaluate the relationship between additive error and relative error, in Figure 6. The observed mean relative error decreases consistently with ϵ; when $\epsilon \leq 0.01$, the relative error is very small (< 3%). A possible concern is whether in extreme cases, a relative-error blows up; based on our results, the empirical relative error often shows a clear monotonic trend with respect to additive error. The curve based on this observed empirical monotonic trend often provides sufficient information to determine reasonable ϵ values for a target relative error.

8.3 Budget-Allocation Sensitivity Study

We study the sensitivity of CANWAS by decomposing the total error budget ϵ into truncation (ϵ_t), projection (ϵ_r), and sampling (ϵ_w) components, which directly determine the hyperparameters $L \propto \ln(1/\epsilon_t)$, $T \propto \epsilon_r^{-2}$, and $W \propto \epsilon_w^{-2}$. We evaluate seven allocation strategies on the MUTAG and Facebook graphs, comparing a uniform split against biased configurations that assign 60%–80% of the budget to a single error source.

Figure 7 summarises the results. The pattern is clear. On MUTAG all variants complete in under 40s, with the 60% projection

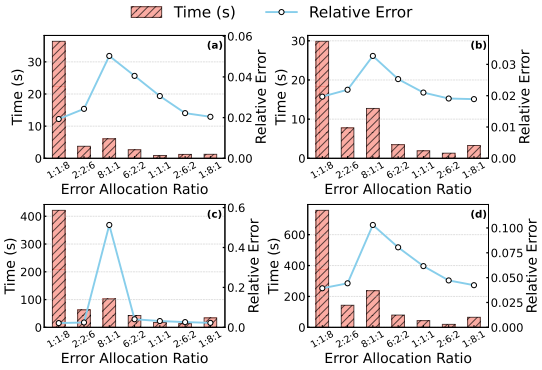


Figure 7: Effect of error budget allocation on two representative graphs: (a) (b) MUTAG and (c) (d) Facebook ⊗ Facebook. The x-axis labels denote the allocation ratio $\epsilon_t : \epsilon_r : \epsilon_w$.

Table 3: Peak memory usage (GB) comparison of methods.

	Direct	CG	FPI	Ark	GVOYS	**CANWAS**
MUTAG	0.008	0.005	0.005	0.004	0.003	**0.002**
ENZYMES	0.019	0.011	0.011	0.008	0.007	**0.003**
DBLP	0.058	0.043	0.043	0.032	0.016	**0.002**
COLLAB	0.131	0.112	0.112	0.071	0.052	**0.005**
GitHub	0.143	0.122	0.122	0.101	0.041	**0.006**
DD	0.911	0.162	0.162	0.112	0.102	**0.011**
Facebook⊗Facebook	–	–	–	2.212	1.823	**0.014**
Facebook⊗YouTube	–	–	–	–	3.341	**1.612**
Facebook⊗Orkut	–	–	–	–	7.131	**4.956**
GrQc⊗GrQc	–	–	–	1.403	0.621	**0.016**
GrQc⊗YouTube	–	–	–	–	3.341	**1.511**
GrQc⊗Orkut	–	–	–	–	7.131	**4.955**

split the clear sweet-spot—0.61s at ϵ=0.10 (left) and 1.31s at ϵ=0.05 (right)—while still keeping the relative error around two percent. Moving to Facebook, whose product graph is five orders of magnitude larger, runtimes rise by roughly one to two orders but the pattern persists: the 60% projection split remains fastest (13s and 19s, respectively) and its error stays within the target budget, whereas plain sampling (allocating 60%–80% to sampling) is 6–15× slower and aggressive hop-truncation (allocating 80% to truncation) overshoots the error bound. The figure thus confirms that the 60% projection split consistently offers the best time–accuracy trade-off.

8.4 Memory Overhead

Table 3 reports the peak resident set size of every contender on the six small graphs and on each of the six large products. Two facts stand out. First, the classic exact solvers are already bulky on the moderate instances—up to 0.21 GB on DD—and cannot be launched at all on the product graphs. Ark and GVOYS cut the footprint substantially, yet Ark still needs 1.4 GB on the homogeneous Facebook⊗Facebook pair and fails to initialise on the four largest cross-network products, while GVOYS rises from 0.62 GB on Facebook ⊗ Facebook to more than 7 GB on Facebook ⊗ Orkut. Second, our method is consistently the most frugal: it stays below 12 MB on every small graph, uses only 14 MB on the homogeneous products, and—even on the billion-scale-node Facebook⊗ Orkut pair—tops out at 5 GB, a 30% saving over the best competing configuration. The results confirm that CANWAS keeps memory requirements essentially linear in the size of the two input graphs.

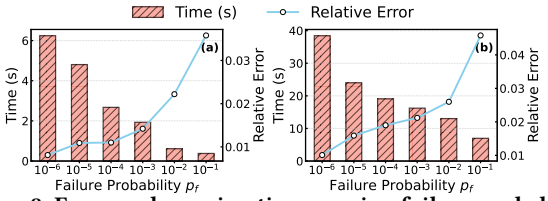


Figure 8: Error and running time varying failure probability on two networks: (a) MUTAG and (b) Facebook ⊗ Facebook.

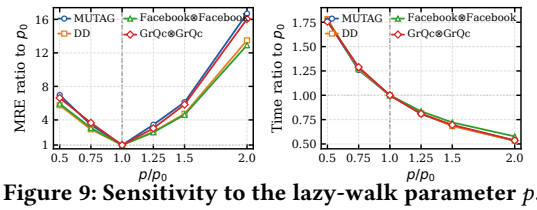


Figure 9: Sensitivity to the lazy-walk parameter p.

8.5 Varying Failure Probability p_f

Figure 8 illustrates the sensitivity of CANWAS to the failure-probability budget. We observe a mild and monotone trade-off where relaxing p_f from $p_f = 10^{-2}$ to 10^{-1} yields only marginal latency reductions. Conversely, enforcing a strict probability of $p_f = 10^{-6}$ increases runtime by approximately 5× while reducing the empirical error by less than 2%. These results confirm the robustness of the algorithm to parameter variations and validate the default setting of $p_f = 0.01$ as an effective balance between efficiency and reliability.

8.6 Sensitivity to Lazy-Walk Parameter p

We vary the lazy-walk parameter around its default value $p_0 = 1 - \sqrt{1-\alpha}$, using $p/p_0 \in \{0.5, 0.75, 1.0, 1.25, 1.5, 2.0\}$ while keeping the target RWK and all other sampling budgets fixed. Figure 9 reports the mean relative error and runtime, normalized by their values at p_0. The default p_0 gives the most stable trade-off: larger p reduces runtime but increases error, while smaller p increases runtime without improving accuracy. This supports using the decay-matched p_0 as the default lazy-walk parameter.

8.7 Case Studies

We give two case studies. **Case Study I** focuses on *graph retrieval* over a brain-network database, demonstrating that CANWAS achieves high ranking accuracy with improved efficiency. **Case Study II** targets the *accelerated training* of graph neural networks [53, 54]. By integrating CANWAS as the core solver, we show that it effectively overcomes the bottleneck of existing iterative methods, enabling faster training without compromising performance.

8.7.1 Case Study I: Brain Graph Retrieval. We evaluate the RWK on the brain graph database[1] derived from the Human Connectome Project [38]. For each network pair (G, H), we set source vector $\boldsymbol{v}$ and target vector $\boldsymbol{w}$ to be uniform over the diagonal pairs (i, i) in the product vertex set $V_\times$. This configuration captures the natural one-to-one correspondence of brain regions, where the resulting scalar $\kappa(\boldsymbol{v}, \boldsymbol{w})$ quantifies the similarity supported by random walks.

We benchmark CANWAS against Direct [21] and GVOYS [15] using identical distributional kernel queries. Treating each brain graph as a query G, we compute $\kappa(\boldsymbol{v}, \boldsymbol{w})$ against all database entries

[1] https://braingraph.org/

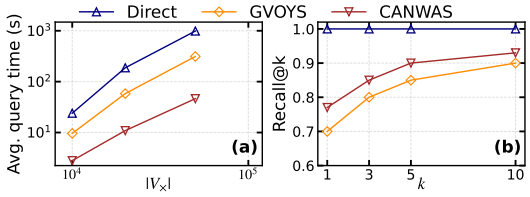


Figure 10: Comparison of cost and recall of different methods.

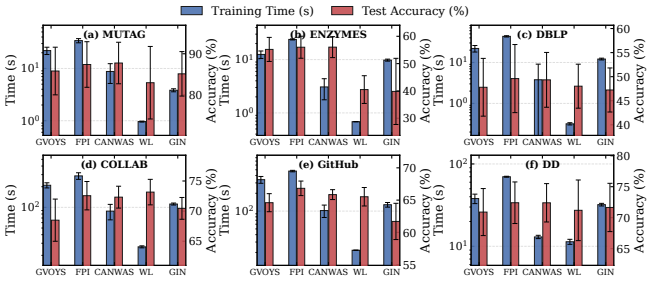


Figure 11: Training time and test accuracy of GRWNN with different RWK evaluators and the WL and GIN baselines.

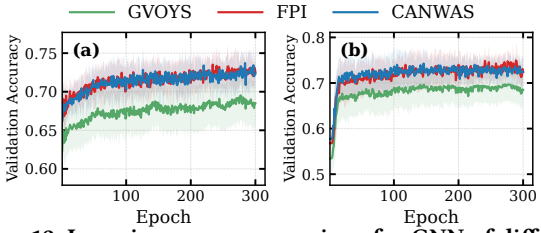


Figure 12: Learning curve comparison for GNN of different methods: (a) COLLAB and (b) DD.

H to generate rankings. Figure 10 shows the kernel computation time and retrieval quality gauged as Recall@k, utilizing subject labels as ground truth. Direct and GVOYS become computationally expensive as graphs grow to hundreds of nodes. In contrast, CANWAS substantially reduces query time while retaining competitive Recall@k across the tested settings.

8.7.2 Case Study II: Accelerating Graph Neural Networks. We validate CANWAS in graph neural networks by integrating it into the GRWNN framework on six graph classification datasets listed in Table 2. We compare against the RWK/RWNN-based baselines FPI and GVOYS, and further include WL and GIN [76] as non-RWK baselines, using the same data splits and evaluation protocol. Figure 11 shows that CANWAS provides a favorable speed–accuracy tradeoff: across the six datasets, CANWAS substantially reduces training time compared with FPI while preserving comparable test accuracy, and it is both faster and more accurate than GVOYS. For example, on DD, CANWAS cuts the training time from 69.98 seconds with FPI to 13.0 seconds, yielding a 5.4× speedup while maintaining nearly the same test accuracy. Compared with WL, CANWAS achieves higher test accuracy on most evaluated datasets, while WL remains faster as a lightweight non-RWK baseline. Compared with GIN, CANWAS achieves higher test accuracy across all six datasets and is faster on most of them. The learning curves in Figure 12 further show the training dynamics on two representative datasets: CANWAS converges to nearly the same final accuracy as FPI, whereas GVOYS consistently remains less accurate. In summary, CANWAS makes the RWK-based approach a compelling alternative to other methods by making it scalable with error guarantees.

9 CONCLUSION

We present the first RWK algorithm that achieves near-linear time and memory complexity while providing a rigorous additive-error guarantee. By combining walk truncation with Rademacher sketches computed independently on each graph, our method avoids constructing the quadratic-sized product graph, and thus efficiently supports both distributional and single-source RWK queries. Empirically, our approach delivers an order-of-magnitude speedup on billion-scale product graphs, with significantly lower memory overhead and strong practical accuracy compared to SOTA methods.

ACKNOWLEDGMENTS

This research / project is supported by the National Research Foundation, Singapore, under its AI Singapore Programme (AISG Award No: AISG3-RP-2024-034), and under its Frontier CRP Grant (NRF-F-CRP-2024-0005). This research is also supported by the Ministry of Education, Singapore, under its MOE AcRF Tier-2 Award (MOE-T2EP20224-0005) and MOE AcRF Tier 3 Award (MOE-MOET32022-0006). Lakshmanan's research was supported in part by a Discovery Grant grant (Grant Number RGPIN-2025-06874) and an Alliance Grant from the Natural Sciences and Engineering Research Council of Canada. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore.

REFERENCES

[1] Dimitris Achlioptas. 2003. Database-friendly random projections: Johnson–Lindenstrauss with binary coins. *J. Comput. System Sci.* 66, 4 (2003), 671–687.
[2] Reid Andersen, Christian Borgs, Jennifer Chayes, John Hopcroft, Vahab S. Mirrokni, and Shang-Hua Teng. 2007. Local computation of PageRank contributions. In *Proceedings of the 5th International Conference on Algorithms and Models for the Web-Graph*. Springer-Verlag, 150–165.
[3] Reid Andersen, Fan Chung, and Kevin Lang. 2006. Local Graph Partitioning using PageRank Vectors. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science*. IEEE Computer Society, 475–486.
[4] Francis R. Bach. 2008. Graph kernels between point clouds. In *Proceedings of the 25th International Conference on Machine Learning*. Association for Computing Machinery, 25–32.
[5] Siddhartha Banerjee and Peter Lofgren. 2015. Fast Bidirectional Probability Estimation in Markov Models. In *Advances in Neural Information Processing Systems*, Vol. 28. Curran Associates, Inc.
[6] Karsten Borgwardt, Nicol Schraudolph, and S.v.n. Vishwanathan. 2006. Fast Computation of Graph Kernels. In *Advances in Neural Information Processing Systems*, B. Schölkopf, J. Platt, and T. Hoffman (Eds.), Vol. 19. MIT Press.
[7] Karsten M. Borgwardt, Sebastian Böttger, and Hans-Peter Kriegel. 2006. VGM: Visual Graph Mining. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*. 733–735.
[8] Karsten M. Borgwardt and Hans-Peter Kriegel. 2005. Shortest-Path Kernels on Graphs. In *Proceedings of the Fifth IEEE International Conference on Data Mining*. IEEE Computer Society, 74–81.
[9] Karsten M Borgwardt, SVN Vishwanathan, and Hans-Peter Kriegel. 2006. Class prediction from time series gene expression profiles using dynamical systems kernels. In *Biocomputing 2006*. World Scientific, 547–558.
[10] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. 2023. Improving Graph Neural Network Expressivity via Subgraph Isomorphism Counting. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 45. 657–668.
[11] Horst Bunke. 2000. Graph matching: Theoretical foundations, algorithms, and applications. In *Proc. Vision Interface*, Vol. 2000. 82–88.
[12] Zhengdao Chen, Lei Chen, Soledad Villar, and Joan Bruna. 2020. Can Graph Neural Networks Count Substructures?. In *Advances in Neural Information Processing Systems*.
[13] Minsu Cho, Jungmin Lee, and Kyoung Mu Lee. 2010. Reweighted random walks for graph matching. In *European conference on computer vision*. Springer, 492–505.
[14] Yun Young Choi, Sun Woo Park, Youngho Woo, and U Jin Choi. 2023. Cycle to Clique (Cy2C) Graph Neural Network: A Sight to See beyond Neighborhood Aggregation. In *International Conference on Learning Representations*.

[15] Krzysztof Marcin Choromanski, Isaac Reid, Arijit Sehanobish, and Kumar Avinava Dubey. 2025. Optimal Time Complexity Algorithms for Computing General Random Walk Graph Kernels on Sparse Graphs. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, Vol. 258. PMLR, 3457–3465.

[16] Luca Cosmo, Giorgia Minello, Alessandro Bicciato, Michael M. Bronstein, Emanuele Rodolà, Luca Rossi, and Andrea Torsello. 2025. Graph Kernel Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 36, 4 (2025), 6257–6270.

[17] Aosong Feng, Chenyu You, Shiqiang Wang, and Leandros Tassiulas. 2022. KerGNNs: Interpretable Graph Neural Networks with Graph Kernels. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 6 (2022), 6614–6622.

[18] Donniell E Fishkind, Sancar Adali, Heather G Patsolic, Lyudmila Meng, Disa Singh, Vince Lyzinski, and Carey E Priebe. 2019. Seeded graph matching. *Pattern recognition* 87 (2019), 203–215.

[19] Dániel Fogaras and Balázs Rácz. 2004. Towards Scaling Fully Personalized PageRank. In *Algorithms and Models for the Web-Graph*, Stefano Leonardi (Ed.). Springer, 105–117.

[20] Judith D. Gardiner, Alan J. Laub, James J. Amato, and Cleve B. Moler. 1992. Solution of the Sylvester matrix equation AXBT + CXDT = E. *ACM Trans. Math. Softw.* 18, 2 (June 1992), 223–231.

[21] Thomas Gärtner, Peter Flach, and Stefan Wrobel. 2003. On Graph Kernels: Hardness Results and Efficient Alternatives. In *Learning Theory and Kernel Machines*. Springer, 129–143.

[22] Thomas Gärtner, John W. Lloyd, and Peter A. Flach. 2004. Kernels and Distances for Structured Data. *Machine Learning* 57, 3 (01 Dec 2004), 205–232.

[23] Swarnendu Ghosh, Nibaran Das, Teresa Gonçalves, Paulo Quaresma, and Mahantapas Kundu. 2018. The journey of graph kernels through two decades. *Computer Science Review* 27 (2018), 88–111.

[24] Frank Harary. 2018. *Graph theory*. CRC Press.

[25] Taher H. Haveliwala. 2002. Topic-sensitive PageRank. In *Proceedings of the 11th International Conference on World Wide Web*. Association for Computing Machinery, 517–526.

[26] Harrie Hendriks and Martien C. A. van Zuijlen. 2017. Linear combinations of Rademacher random variables. *arXiv: Combinatorics* (2017).

[27] Shohei Hido and Hisashi Kashima. 2009. A Linear-Time Graph Kernel. In *Proceedings of the 2009 Ninth IEEE International Conference on Data Mining*. 179–188.

[28] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30.

[29] Tamás Horváth, Thomas Gärtner, and Stefan Wrobel. 2004. Cyclic pattern kernels for predictive graph mining. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 158–167.

[30] Sergey Ivanov and Evgeny Burnaev. 2018. Anonymous Walk Embeddings. In *Proceedings of the 35th International Conference on Machine Learning*, Vol. 80. PMLR, 2186–2195.

[31] Glen Jeh and Jennifer Widom. 2003. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web*. Association for Computing Machinery, 271–279.

[32] Hyundoo Jeong and Byung-Jun Yoon. 2015. Accurate multiple network alignment through context-sensitive random walk. *BMC Systems Biology* 9, Suppl 1 (2015), S7.

[33] Xinpeng Jiang, Haoyu Liu, Siqiang Luo, and Xiaokui Xiao. 2026. Near-Optimality for Single-Source Personalized PageRank. *Proceedings of the ACM on Management of Data* 4, 2 (PODS) (2026), 1–21.

[34] Charilaos Kanatsoulis and Alejandro Ribeiro. 2024. Counting Graph Substructures with Graph Neural Networks. In *International Conference on Learning Representations*.

[35] U Kang, Hanghang Tong, and Jimeng Sun. 2012. Fast Random Walk Graph Kernel. In *Proceedings of the 2012 SIAM International Conference on Data Mining (SDM)*. 828–838.

[36] Hisashi Kashima, Koji Tsuda, and Akihiro Inokuchi. 2003. Marginalized kernels between labeled graphs. In *Proceedings of the Twentieth International Conference on International Conference on Machine Learning*. AAAI Press, 321–328.

[37] Ehsan Kazemi, S Hamed Hassani, and Matthias Grossglauser. 2015. Growing a graph matching from a handful of seeds. *Proceedings of the VLDB Endowment* 8, 10 (2015), 1010–1021.

[38] Csaba Kerepesi, Balázs Szalkai, Bálint Varga, and Vince Grolmusz. 2017. The braingraph.org Database of High Resolution Structural Connectomes and the Brain Graph Tools. *Cognitive Neurodynamics* 11, 5 (2017), 483–486.

[39] Risi Kondor, Nino Shervashidze, and Karsten M. Borgwardt. 2009. The graphlet spectrum. In *Proceedings of the 26th Annual International Conference on Machine Learning*. Association for Computing Machinery, 529–536.

[40] Jure Leskovec and Rok Sosič. 2016. SNAP: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology* 8, 1 (2016), 1.

[41] Jiangneng Li, Haitao Yuan, Gao Cong, Han Mao Kiah, and Shuhao Zhang. 2025. MAST: Towards Efficient Analytical Query Processing on Point Cloud Data. *Proc. ACM Manag. Data* 3, 1 (2025), 27.

[42] Xin Li and Hsinchun Chen. 2013. Recommendation as Link Prediction in Bipartite Graphs: A Graph Kernel-Based Machine Learning Approach. *Decision Support Systems* 54, 2 (2013), 880–890.

[43] Meihao Liao, Rong-Hua Li, Qiangqiang Dai, and Guoren Wang. 2022. Efficient Personalized PageRank Computation: A Spanning Forests Sampling Based Approach. In *Proceedings of the 2022 International Conference on Management of Data*. Association for Computing Machinery, 2048–2061.

[44] Ningyi Liao, Dingheng Mo, Siqiang Luo, Xiang Li, and Pengcheng Yin. 2022. SCARA: scalable graph neural networks with feature-oriented optimization. *Proc. VLDB Endow.* 15, 11 (2022), 3240–3248.

[45] Changan Liu, Haoxin Sun, Ahad N. Zehmakan, and Zhongzhi Zhang. 2026. Efficient edge rewiring strategies for enhancing PageRank fairness. *Theoretical Computer Science* 1067 (2026), 115765.

[46] Changan Liu, Zixuan Xie, Ahad N. Zehmakan, and Zhongzhi Zhang. 2026. Efficient Algorithms for Computing Random Walk Centrality. *IEEE Transactions on Knowledge and Data Engineering* 38, 1 (2026), 235–247.

[47] Changan Liu, Ahad N. Zehmakan, and Zhongzhi Zhang. 2024. Fast Query of Biharmonic Distance in Networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 1887–1897.

[48] Changan Liu, Xinna Zhou, Bo Zhang, Ahad N. Zehmakan, and Zhongzhi Zhang. 2026. Fast Estimation of Pairwise Biharmonic Distance on Graphs. *Proceedings of the ACM on Management of Data* 4, 2 (2026), 27.

[49] Haoyu Liu, Ningyi Liao, and Siqiang Luo. 2025. SIGMA: An efficient heterophilous graph neural network with fast global aggregation. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1924–1937.

[50] Haoyu Liu and Siqiang Luo. 2024. Bird: Efficient approximation of bidirectional hidden personalized pagerank. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2255–2268.

[51] Pierre Mahé, Nobuhisa Ueda, Tatsuya Akutsu, Jean-Luc Perret, and Jean-Philippe Vert. 2005. Graph Kernels for Molecular Structure-Activity Relationship Analysis with Support Vector Machines. *Journal of Chemical Information and Modeling* 45, 4 (2005), 939–951.

[52] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*. arXiv:2007.08663 www.graphlearning.io

[53] Giannis Nikolentzos and Michalis Vazirgiannis. 2020. Random Walk Graph Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 16211–16222.

[54] Giannis Nikolentzos and Michalis Vazirgiannis. 2023. Geometric Random Walk Graph Neural Networks via Implicit Layers. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, Vol. 206. PMLR, 2035–2053.

[55] Jorge Nocedal and Stephen J. Wright. 2006. *Numerical optimization*. Springer, New York, NY.

[56] Vito Claudio Ostuni, Tommaso Di Noia, Roberto Mirizzi, and Eugenio Di Sciascio. 2014. A Linked Data Recommender System Using a Neighborhood-Based Graph Kernel. In *Proceedings of the 15th International Conference on Electronic Commerce and Web Technologies (EC-Web)*, Vol. 188. Springer, 89–100.

[57] K. B. Petersen and M. S. Pedersen. 2012. The Matrix Cookbook.

[58] Chengzhi Piao, Tingyang Xu, Xiangguo Sun, Yu Rong, Kangfei Zhao, and Hong Cheng. 2023. Computing Graph Edit Distance via Neural Graph Matching. *Proc. VLDB Endow.* 16, 8 (2023), 1817–1829.

[59] Liva Ralaivola, Sanjay J. Swamidass, Hiroto Saigo, and Pierre Baldi. 2005. Graph kernels for chemical informatics. *Neural Networks* 18, 8 (2005), 1093–1110.

[60] Radu Bogdan Rusu, Nico Blodow, and Michael Beetz. 2009. Fast point feature histograms (FPFH) for 3D registration. In *2009 IEEE International Conference on Robotics and Automation*. IEEE, 3212–3217.

[61] Hiroto Saigo, Masahiro Hattori, Hisashi Kashima, and Koji Tsuda. 2010. Reaction graph kernels predict EC numbers of unknown enzymatic reactions in plant secondary metabolism. *BMC Bioinformatics* 11, 1 (2010), S31.

[62] Nino Shervashidze and Karsten Borgwardt. 2009. Fast subtree kernels on graphs. In *Advances in Neural Information Processing Systems*, Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta (Eds.), Vol. 22. Curran Associates, Inc.

[63] N. Shervashidze, SVN. Vishwanathan, TH. Petri, K. Mehlhorn, and KM. Borgwardt. 2009. Efficient Graphlet Kernels for Large Graph Comparison, In JMLR Workshop and Conference Proceedings Volume 5: AISTATS 2009. *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, 488–495.

[64] Jieming Shi, Renchi Yang, Tianyuan Jin, Xiaokui Xiao, and Yin Yang. 2019. Real-time top-k personalized PageRank over large graphs on GPUs. *Proc. VLDB Endow.* 13, 1 (2019), 15–28.

[65] Rohit Singh, Jinbo Xu, and Bonnie Berger. 2008. Global alignment of multiple protein interaction networks with application to functional orthology detection. *Proceedings of the National Academy of Sciences* 105, 35 (2008), 12763–12768.

[66] Mahito Sugiyama and Karsten Borgwardt. 2015. Halting in Random Walk Kernels. In *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence,

D. Lee, M. Sugiyama, and R. Garnett (Eds.), Vol. 28. Curran Associates, Inc.

[67] Hanghang Tong, Christos Faloutsos, and Jia-Yu Pan. 2006. Fast Random Walk with Restart and Its Applications. In *Proceedings of the Sixth International Conference on Data Mining*. IEEE Computer Society, 613–622.

[68] S.V.N. Vishwanathan, Nicol N. Schraudolph, Risi Kondor, and Karsten M. Borgwardt. 2010. Graph Kernels. *Journal of Machine Learning Research* 11, 40 (2010), 1201–1242.

[69] S. V. N. Vishwanathan, Nicol N. Schraudolph, Risi Kondor, and Karsten M. Borgwardt. 2010. Graph Kernels. *J. Mach. Learn. Res.* 11 (2010), 1201–1242.

[70] Sibo Wang, Youze Tang, Xiaokui Xiao, Yin Yang, and Zengxiang Li. 2016. HubPPR: effective indexing for approximate personalized PageRank. *Proc. VLDB Endow.* 10, 3 (2016), 205–216.

[71] Sibo Wang, Renchi Yang, Xiaokui Xiao, Zhewei Wei, and Yin Yang. 2017. FORA: Simple and Effective Approximate Single-Source Personalized PageRank. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 505–514.

[72] Xiaohong Wang, Aaron Smalter, Jun Huan, and Gerald H. Lushington. 2009. G-Hash: Towards Fast Kernel-Based Similarity Search in Large Graph Databases. In *Proceedings of the 12th International Conference on Extending Database Technology (EDBT)*. 472–480.

[73] Zhewei Wei, Xiaodong He, Xiaokui Xiao, Sibo Wang, Shuo Shang, and Ji-Rong Wen. 2018. TopPPR: Top-k Personalized PageRank Queries with Precision Guarantees on Large Graphs. In *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, 441–456.

[74] Paul M. Weichsel. 1962. The Kronecker Product of Graphs. *Proc. Amer. Math. Soc.* 13, 1 (1962), 47–52.

[75] Hao Wu, Junhao Gan, Zhewei Wei, and Rui Zhang. 2021. Unifying the Global and Local Approaches: An Efficient Power Iteration with Forward Push. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, 1996–2008.

[76] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.

[77] Xifeng Yan, Hong Cheng, Jiawei Han, and Philip S. Yu. 2008. Mining Significant Graph Patterns by Leap Search. In *Proc. of SIGMOD*. 433–444.

[78] Mingji Yang, Hanzhi Wang, Zhewei Wei, Sibo Wang, and Ji-Rong Wen. 2024. Efficient Algorithms for Personalized PageRank Computation: A Survey. *IEEE Trans. on Knowl. and Data Eng.* 36, 9 (2024), 4582–4602.

[79] Renchi Yang and Jing Tang. 2023. Efficient Estimation of Pairwise Effective Resistance. *Proc. ACM Manag. Data* 1, 1, Article 16 (2023), 27 pages.

[80] Junda Ye, Zhongbao Zhang, Li Sun, and Siqiang Luo. 2025. MoSE: Unveiling Structural Patterns in Graphs via Mixture of Subgraph Experts. *arXiv preprint arXiv:2509.09337* (2025).

[81] Cai Zhou, Xiyuan Wang, and Muhan Zhang. 2023. Distance-Restricted Folklore Weisfeiler-Leman GNNs with Provable Cycle Counting Power. In *Advances in Neural Information Processing Systems*.

[82] Yuanyuan Zhu, Jeffrey Xu Yu, and Lu Qin. 2014. Leveraging Graph Dimensions in Online Graph Search. *Proceedings of the VLDB Endowment* 8, 1 (2014), 85–96.